**Omar Al-Mukhtar University**

**Faculty of Engineering**

**Department of Computer**

# Implementation of Perceptron on FPGAs

| Name | ID |
|---|---|
| **Amira Jomaa Ebrahim** | **20204018** |
| **Amna Alhassan Hamad** | **20204081** |
| **Azizah Ebrahim Edris** | **20194225** |
| **Monya Ashraf Fadel** | **20204186** |
| **Noor Al-huda Miloud Mohammed** | **20204189** |
| **Rbah Masoud Saad** | **20204222** |
| **Souad Abdulatif Saleh** | **20204217** |

**Supervised by: M.Eng. Salem Ali Al-Akaer**

**2026**

# ABSTRACT

Machine learning techniques have gained significant attention in solving classification and decision-making problems across various engineering applications. Among these techniques, the Perceptron algorithm represents one of the earliest and simplest forms of artificial neural networks capable of learning linearly separable patterns. This study investigates the software implementation of the Perceptron algorithm for modeling fundamental digital logic gates and evaluates its performance in comparison with a dedicated hardware realization.

Instead of relying solely on software, a hardware-based Perceptron architecture is designed and implemented on a Field-Programmable Gate Array (FPGA) platform using Verilog Hardware Description Language (HDL). The design is evaluated across both behavioral and structural configurations for different input dimensions (2, 4, and 6 inputs). The performance of the algorithm is assessed in terms of classification accuracy, computational behavior, resource utilization, and power consumption. The post-implementation timing simulations demonstrate that the structural Perceptron architecture achieves successful learning convergence within 55 epochs, with a total convergence time of approximately 64.66 ms.

A comprehensive comparative analysis highlights the trade-offs between software flexibility and the high-speed, power-aware execution of hardware-based digital realizations.

# TABLE OF CONTENTS

TABLE OF CONTENTS ..... i
LIST OF TABLES ..... iii
LIST OF FIGURES ..... iv
LIST OF ABBREVIATIONS ..... v
**CHAPTER 1 INTRODUCTION** ..... 1
1.1 Overview ..... 1
1.2 Problem statement ..... 1
1.3 Aim and objectives ..... 2
1.4 Scope of the project ..... 2
1.5 Contributions ..... 3
1.6 Report organization ..... 3
**CHAPTER 2 LITERATURE REVIEW** ..... 4
2.1 Overview ..... 4
2.2 Machine Learning ..... 4
2.2.1 Learning Algorithms: Perceptron Rule vs. Backpropagation ..... 4
2.2.2 The Artificial Neural Networks and Activation Functions ..... 5
2.2.2.1 Unit Step Function (Heaviside Step) ..... 8
2.3 Field programmable gate array (FPGA) ..... 8
2.4 Machine Learning algorithms on FPGA and CPU ..... 10
2.5 Comparative overview of related works ..... 11
2.6 Summary ..... 12
**CHAPTER 3 METHODOLOGY** ..... 13
3.1 Overview ..... 13
3.2 Flow of the project ..... 13
3.3 Algorithm on CPU ..... 14
3.3.1 The Implementation Algorithm ..... 14
3.3.2 Dataset Representation and Input Scalability ..... 14
3.3.3 Training Process and Parameter Optimization ..... 15
3.3.4 Weight and Bias Update Rule ..... 15
3.4 FPGA Design and Implementation ..... 16
3.4.1 Perceptron Training Algorithm ..... 16
3.4.2 Behavioral Modeling ..... 17

3.4.3 Structural Modeling and Hardware Architecture ..... 25
3.4.3.1 Control Unit (CU) and Finite State Machine (FSM) ..... 26
3.4.3.2 Datapath Unit (DU) ..... 30
3.4.3.3 Timing Synchronization Between CU and DU ..... 33
3.4.4 Lut- based design ..... 34
3.5 Fixed point representation ..... 35
3.5.1 Motivation of the fixed point ..... 36
3.5.2 Representation of Weights and Bias ..... 36
3.5.3 Range of Q6.26 Fixed-Point Format ..... 39
3.5.4 Hardware efficiency impact ..... 39
**CHAPTER 4 RESULTS** ..... 41
4.1 Overview ..... 41
4.2 CPU-Based Implementation and Procedural Logic ..... 41
4.2.1 Modular Organization ..... 41
4.2.2 Implementation Verification (Case Study: n = 6) ..... 42
4.3 Convergence Analysis ..... 43
4.4 Learning Rate Optimization and Sensitivity Analysis ..... 44
4.5 FPGA Implementation and Hardware Results ..... 46
4.5.1 Behavioral model result and analysis ..... 46
4.5.1.1 Run Post-Implementation Timing Simulation ..... 46
4.5.1.2 Behavioral Resource Utilization and Power Analysis ..... 50
4.5.2 Structural results and analysis ..... 53
4.5.2.1 Run Post-Implementation Timing Simulation ..... 53
4.5.2.2 Structural Resource Utilization and Power Analysis ..... 57
4.5.3 Comparison Between Behavioral and Structural Implementations ..... 61
4.5.4 Performance Evaluation and Comparative Analysis ..... 62
**CHAPTER 5 CONCLUSION** ..... 65
5.1 Overview ..... 65
5.2 Conclusion ..... 65
5.3 Future works ..... 66

**CHAPTER 6 REFERENCES** ..... 68

# LIST OF TABLES

Table 2.1: Comparative Overview Of Related Works ..... 11
Table 3.1: Rtl-Cs Table For Structural Perceptron Trainer ..... 29
Table 3.2: Fixed-Point Representation Examples ..... 38
Table 3.3: Range Of Q6.26 Fixed-Point Representation ..... 39
Table 4.1: Impact Of Learning Rate On Convergence (N = 5) ..... 45
Table 4.2: Resource Utilization And Power Consumption Vs. Number Of Inputs. ..... 51
Table 4.3: Resource Utilization And Power Consumption Of The Structural Perceptron Architecture. ..... 60
Table 4.4: Resource And Power Utilization Comparison Between Behavioral And Structural Implementations (6-Inputs). ..... 63
Table 4.5: Execution Time And Performance Comparison Between Software And Hardware Environments. ..... 65

## LIST OF FIGURES

Figure 2.1: General Artificial Neural Network Model ..... 6
Figure 2.2: Representation Of The Unit Step Activation Function ..... 8
Figure 2.3 : The Basic Architecture Of The Fpga. ..... 9
Figure 2.4: The Lut Operates And The Lb Structure ..... 9
Figure 2.5: Programmable Switch Block ..... 10
Figure 3.1: Finite State Machine (Fsm) Description For Behavioral Model ..... 19
Figure 3.2: Behavioral Asm Flow Chart, Part 1 (Generation Data Level) ..... 21
Figure 3.3: Behavioral Asm Flow Chart, Part 2 (Training And Update (Weights And Bias) Level) ..... 22
Figure 3.4: Behavioral Asm Flowchart, Part 3 (Dataset Traversal And Completion Check Level ..... 23
Figure 3.5: Algorithmic State Machine (Asm) Chart Describing The Sequential Execution Flow And State Transitions Of The Hardware Perceptron Trainer. ..... 26
Figure 3.6: Fsm State Transition And Transition Conditions. ..... 29
Figure 3.7: Detailed Block Diagram Of The Datapath Unit (Du). ..... 32
Figure 3.8: Structural Hardware Architecture Showing The Interaction Between The Control Unit And The Datapath Unit. ..... 33
Figure 3.9: Idea Of Fixed-Point Numbers ..... 35
Figure 4.1: Cpu Training Log And Output For A 6-Input And Gate Configuration ..... 41
Figure 4.2: Convergence Behavior Comparison For Different Logic Gates At $n$=6. ..... 43
Figure 4.3: Impact Of Scaling Input Dimensions On And Gate Convergence. ..... 44
Figure 4.4 : results for 2- inputs behavioral design.. ..... 47
Figure 4.5 : results for 4- inputs behavioral design.. ..... 48
Figure 4.6 : results for 6- inputs behavioral design.. ..... 49
Figure 4.7 : results for 8- inputs behavioral design.. ..... 50
Figure 4.8 : On-Chip Power Consumption Distribution for Different Input Sizes (n = 2, 4, 6 and 8) behavioral design ..... 53
Figure 4.9 : Post-implementation timing simulation results for the proposed perceptron architecture for 2 inputs ..... 55
Figure 4.10 : Post-implementation timing simulation results for the proposed perceptron architecture for 4 inputs.. ..... 56
Figure 4.11 : Post-implementation timing simulation results for the proposed perceptron architecture for 6 inputs.. ..... 57

Figure 4.12 : Post-implementation timing simulation results for the proposed perceptron architecture for 8 inputs.. ..... 58
Figure 4.13 : On-Chip Power Consumption Distribution for Different Input Sizes (n = 2, 4, 6 and 8).. ..... 61

## LIST OF ABBREVIATIONS

| | | |
|---|---|---|
| ANN | - | Artificial Neural Network |
| GPU | - | Graphics Processing Unit |
| CPU | - | Central Processing Unit |
| FPGA | - | Field-programmable gate array |
| HLS | - | High-level Synthesis |
| DSP | - | Digital Signal Processing |
| LB | - | Logic Blocks |
| LUT | - | Lookup Table |
| MLP | - | Multi-Layer Perceptron |

# CHAPTER 1
# INTRODUCTION

## 1.1 Overview

This project focuses on the implementation of artificial neural networks (ANNs) on hardware, specifically using Field Programmable Gate Arrays (FPGAs). Unlike traditional approaches that rely on complex multi-layer neural networks, this work employs a Single Layer Perceptron (SLP) to perform fundamental logic operations.

FPGAs are reconfigurable logic devices that can be programmed to perform customized computations with high execution speed, approaching that of dedicated hardware. They offer significant advantages such as parallel processing, flexibility, and the ability to adapt system configurations even after deployment. These features make FPGAs highly suitable for implementing machine learning models, particularly in applications that require fast and efficient computation.

Artificial neural networks are inspired by the structure and functionality of the human brain and are widely used in applications such as pattern recognition, classification, and prediction. However, implementing complex neural networks on hardware can be resource-intensive. Therefore, this project adopts a simplified Perceptron model, which requires minimal computational resources while maintaining effective performance for linearly separable problems.

## 1.2 Problem statement

Implementing machine learning models on hardware platforms such as FPGAs presents several challenges, including high computational complexity, increased resource utilization, and longer execution time, especially when using complex learning algorithms such as Backpropagation.

In addition, traditional digital logic design relies on fixed logic gates (AND, OR, NAND, NOR), which lack flexibility and scalability. These limitations motivate the need for a more adaptive and efficient approach that can combine learning capabilities with hardware efficiency.

## 1.3 Aim and objectives

The main aim of this project is to design and implement a Perceptron-based adaptive logic system on an FPGA using efficient hardware techniques.

The objectives of this project are:

- Implement the Perceptron algorithm using Python on a CPU.
- Generate datasets based on truth tables for different logic gates.
- Train the model and extract optimized weights and bias values.
- Implement the trained model on an FPGA using Verilog.
- Replace traditional logic gates with an adaptive learning-based approach.
- Evaluate system performance in terms of speed, resource utilization, and power consumption.
- Compare the accuracy and speedup performance between FPGA and CPU.

## 1.4 Scope of the project

This project focuses on the implementation of a Single Layer Perceptron for fundamental logic gate operations such as AND, OR, NAND, and NOR. The design is implemented on an FPGA using Verilog, with emphasis on hardware efficiency and scalability.

The system utilizes fixed-point arithmetic instead of floating-point to reduce complexity and improve performance. The design also supports scalable input sizes, allowing the model to handle higher-dimensional input configurations.

## 1.5 Contributions

This project introduces an adaptive logic design approach by replacing traditional fixed logic gates with a learning-based Perceptron model. The main contributions include:

- Implementing logic functions using a machine learning model instead of conventional gates.
- Utilizing an FPGA LUT-based architecture for efficient hardware realization.
- Reducing hardware complexity by using shift operations instead of multipliers.
- Applying fixed-point representation to improve performance and reduce resource usage.
- Developing a scalable system capable of handling multiple input configurations.
- This approach enhances flexibility, efficiency, and performance compared to traditional logic design methods.

## 1.6 Report organization

This report is structured into six main chapters. Chapter 2 provides a comprehensive literature review, covering fundamental concepts in machine learning and reviewing related works on FPGA-based implementations. Chapter 3 presents the proposed methodology, detailing both the software modeling and the hardware design of the perceptron system. Chapter 4 discusses the experimental results, including performance evaluation, resource utilization, and power analysis for both behavioral and structural implementations. Chapter 5 concludes the report by summarizing the key findings and suggesting potential directions for future work and system enhancements. Finally, Chapter 6 lists the references cited throughout this report.

# CHAPTER 2
# LITERATURE REVIEW

## 2.1 Overview

Artificial Neural Networks (ANNs) are designed to process information in a manner inspired by the biological brain. In this research, the focus is specifically on the Single Layer Perceptron (SLP). The SLP serves as the simplest form of a neural network, consisting of an input layer, adjustable weights, a bias, and an activation function. This architecture is highly effective for implementing fundamental logic operations on FPGA hardware due to its low computational complexity and high processing speed.

## 2.2 Machine Learning

Machine Learning is a subset of Artificial Intelligence that focuses on the development of algorithms capable of learning from data and making predictions. Unlike traditional rule-based programming, ML models identify underlying patterns through iterative training processes. In the development of neural processors, ML serves as the optimization engine for determining the connection strengths, or weights, within the network architecture.

For this project, a Supervised Learning paradigm was implemented in a Python environment to train the Perceptron model. By feeding the system a labeled dataset (Truth Tables), the algorithm autonomously adjusts the weights and bias until the error is minimized. This ensures that the logical behavior of the neuron is perfectly modeled in software before its final synthesis into hardware logic.

### 2.2.1 Learning Algorithms: Perceptron Rule vs. Backpropagation

In the development of neural networks, choosing the appropriate learning algorithm is a critical design decision that affects both accuracy and hardware

efficiency. In this research, two primary algorithms were evaluated: the Perceptron Learning Rule and Backpropagation.

The Backpropagation algorithm is widely recognized for its ability to train complex multi-layer networks by calculating gradients and updating weights across hidden layers. However, implementing Backpropagation directly for hardware acceleration on an FPGA presents significant challenges. It requires high computational precision, continuous activation functions (like Sigmoid), and extensive use of memory and DSP slices, which can lead to increased latency and resource consumption.

On the other hand, the Perceptron Learning Rule was selected for this project. Since the target of this implementation is Adaptive Logic Gates—which are linearly separable—the Perceptron rule provides an optimal balance. Its mathematical simplicity, relying on basic arithmetic without the need for complex derivatives, makes it highly Hardware-Friendly. By using this rule, we achieved the required intelligence for the logic gates while maintaining maximum processing speed and minimum area overhead on the FPGA chip.

### 2.2.2 The Artificial Neural Networks and Activation Functions

To get a clear idea of the main elements used to construct ANN models, in Fig. 2.1, we provide a general artificial neural network model that contains the main components for this type of model. $x_1, \ldots, x_p$ represents the information (input) that the neuron receives from the external sensory system or from other neurons with which it has a connection. w = $(w_1, \ldots, w_p)$ is the vector of synaptic weights that modifies the received information, emulating the synapse between the biological neurons.

These can be interpreted as gains that can attenuate or amplify the values that they wish to propagate toward the neuron. Parameter b is known as the bias (intercept or threshold) of a neuron. Here in ANN, learning refers to the method of modifying the weights of connections between the nodes (neurons) of a specified network. The different values that the neuron receives are modified by the synaptic weights, which are then added together to produce what is called the net input. In mathematical notation, that is equal to:

$$v = \sum_{i=1}^{P} (\omega_i \cdot x_i) + b$$

This net input ($v$) is what determines whether the neuron is activated or not. The activation of the neuron depends on what we call the activation function. The net input is evaluated in this function, and we obtain the output of the network as shown next:

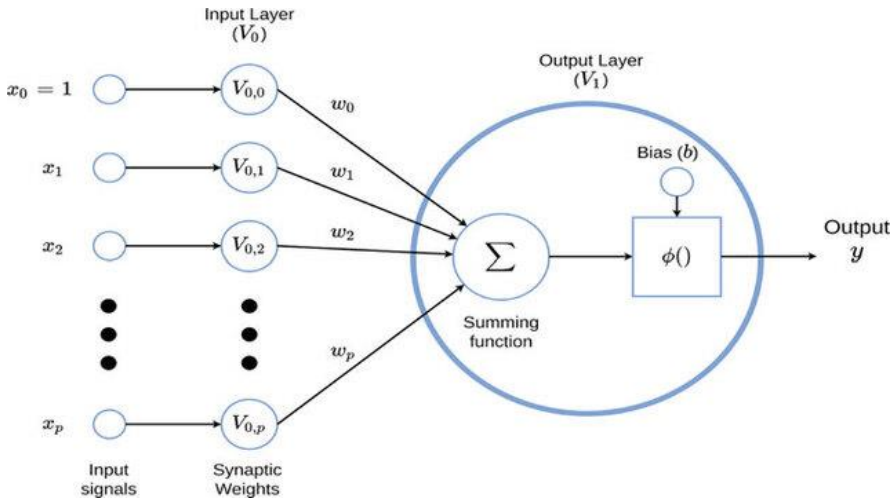



Figure 2.1: General Artificial Neural Network Model

$$y = g(v)$$

Where $g$ is the activation function. For example, if we define this function as a unit step (threshold), the output $y$ will be 1 if the net input $v$ is greater than zero; otherwise, the output will be 0. The generated output $y$ can then be propagated to other neurons or serve as the final output of the network.

To achieve the desired logical behavior, the network must undergo a training process. This is governed by the Perceptron Learning Rule, which iteratively adjusts the weights and bias to minimize the error between the actual and target outputs. The adjustment is based on the following parameters:

Target Output (d): The desired result from the logic gate's truth table.
Actual Output (y): The result produced by the activation function g(v).
Error Signal (e): Defined as the difference between the target and actual output:

$$\boldsymbol{e = d - y}$$

Learning Rate ($\eta$): A constant that determines the step size of each update. A small $\eta$ ensures stable convergence.
The weights are updated according to the Delta Rule. For each input ($x_i$), the new weight value at iteration (t+1) is calculated as follows:

$$\boldsymbol{w_i(t+1) = w_i(t) + \Delta w_i}$$

Where the correction term ($\Delta w_i$) is defined by:

$$\boldsymbol{\Delta w_i = \eta \cdot (d - y) \cdot x_i}$$

Similarly, the Bias (b) is updated to shift the decision threshold:

$$\boldsymbol{b(t+1) = b(t) + \eta \cdot (d - y)}$$

This process continues through multiple Epochs (training cycles) until the weights stabilize and the system achieves Zero Error for all input combinations. Once the training in the software environment (Python) is complete, these optimized weights are extracted and implemented as fixed constants within the FPGA hardware architecture.

#### 2.2.2.1 Unit Step Function (Heaviside Step)

The Unit Step function (also known as the threshold function) is the most suitable activation function for implementing basic logic gates. It produces a binary output based on the net input. If the net input reaches a certain

threshold (usually zero), the neuron fires and produces an output of 1; otherwise, it remains 0. Mathematically, it is defined as:

$$g(v) = \begin{cases} \mathbf{1} \ \ \boldsymbol{if}\ \boldsymbol{v} > \mathbf{0} \\ \mathbf{0} \ \ \boldsymbol{if}\ \boldsymbol{v} \leq \mathbf{0} \end{cases}$$

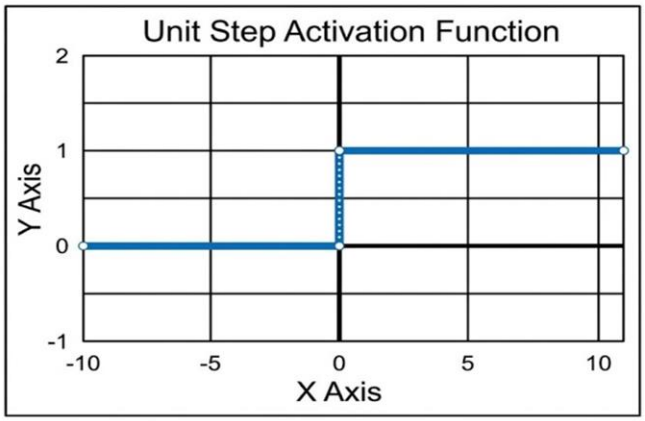



Figure 2.2: Representation of the Unit Step activation function

This binary behavior directly emulates the high and low states of digital logic gates, making it the ideal choice for FPGA-based hardware implementations.

## 2.3 Field programmable gate array (FPGA)

An FPGA is a device constructed with semiconductor materials. FPGA consists of a large number of blocks (LB) connected using configurable interconnections. Logical gates such as AND and XOR gates can be simply implemented by LBs. FPGA also consists of routing channels. These routing channels are running between logic blocks, which are programmed to make the LB interact with each other. The basic architecture of an FPGA is shown in Figure 2.3.

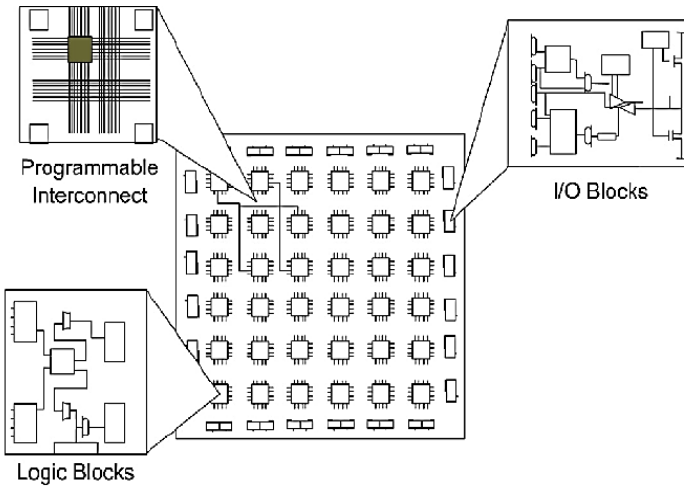



Figure 2.3 : The basic architecture of the FPGA .

As the technology improves, there are circuits that are designed and embedded inside the FPGA to perform specific tasks such as multiplication and digital signal processing DSP.

As mentioned before, the basic component in an FPGA is an LB which has a huge number of logical blocks embedded inside the chip. Lookup Table (LUT) is performed inside LB, which usually contains four inputs. There is an example in Figure 2.4 (a) that shows where the LUT exists. Essentially, LB is defined by this truth table. Recently, designers have added more input pins to increase the speed.

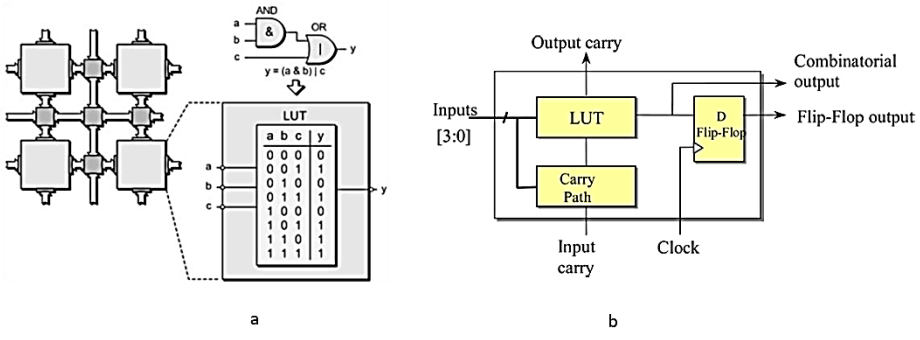



Figure 2.4: The LUT operates and the LB structure

The structures of LB include an even number of logic units. These logic units are the building blocks of the brain of the FPGA. The logic unit body is presented in Figure 2.4 (b). LUT with 4 inputs is inside the LB. The LUT can be programmed to design a combinational function or a memory. In addition, there is a carry look-ahead data path. When mathematical formulas need to be performed, the carry look-ahead will be used to give accurate results. Finally, an FPGA consists of a D-type flip-flop. A D-type flip-flop consists of control signals such as enable set, and reset, used in asynchronous or synchronous modes. Sometimes it is necessary to give permission for the output to be accessed, depending on the FSM, because the inputs of any LB might be its own output.

LBs are connected using routing channels. At the routing channel's heart, there are programmable switch blocks that allow certain crossing wires to be connected. A huge number of configurable logic blocks allow the functionality of the FPGA to be implemented. Figure 2.5 shows the switch blocks.

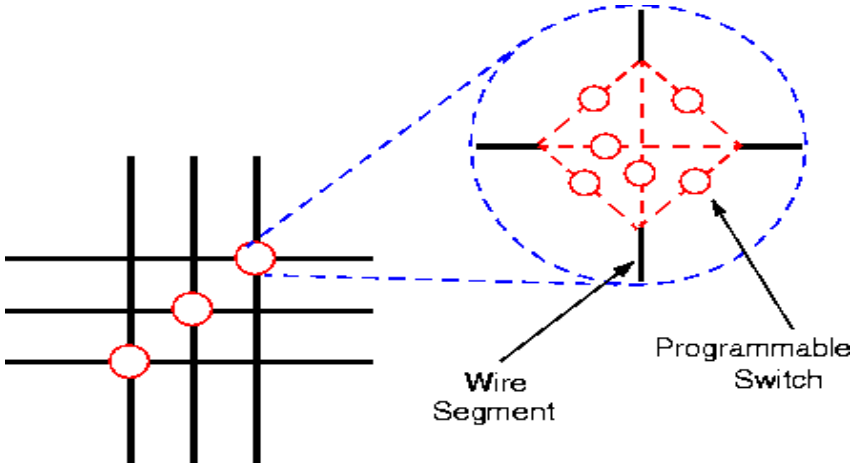



Figure 2.5: Programmable Switch Block

## 2.4 Machine Learning algorithms on FPGA and CPU

In [9], over the last few years, there was a race between CPU and FPGA vendors in terms of the amount of computation that can be handled efficiently at high speed. In addition, the popularity of machine learning and its complex

computation demands makes it one of the important comparisons between CPU and FPGA.

In [1], this design solution eliminates the FPGA design for an artificial neural network. The motivation for this study stems from the fact that an FPGA coprocessor with limited logic density and capabilities can be used in building artificial neural networks, which are widely used in solving different problems.

In [14], the discussion began with the introduction of popular FPGA platforms for machine learning, which include Xilinx Alveo from AMD and Arria from Intel. The comparison between GPUs and CPUs revealed that FPGAs offer high parallelism and customizability, but also require a higher level of design complexity.

## 2.5 Comparative overview of related works

The main papers that this research is based on and closely related to it are summarized in Table 2.1.

Table 2-1 Comparative overview of related works

| No. | Title | Authors/year | Description | Findings |
|---|---|---|---|---|
| 1. | Implementation of Artificial Neural Networks on FPGAs | Hafsa Soliman Mohmed, Nada Faisal Salem, Wessal Mustafa Mohammed,Maryam Abd Almunem Saeid, Retaj Ifdial Omar, Hajer Mohammed Ali, Huda Emrage Mohammed.<br><br>2024 | This study presents the implementation of an Artificial Neural Network (ANN) on FPGA hardware for digit recognition using binary image matrices. The model is first trained and evaluated in MATLAB on CPU, then implemented in FPGA using Verilog. | The results show that FPGA achieves significantly faster execution (up to 4615× faster than CPU) while maintaining comparable accuracy, demonstrating the efficiency of hardware acceleration for ANN applications. |

| No. | Title | Authors/year | Description | Findings |
|---|---|---|---|---|
| 2. | FPGA Design Methodology for Industrial<br><br>Control Systems--A Review | Eric Monmasson, Marcian N. Cirstea<br><br>2007 | Overview on advantages general architecture and software development of FPGA<br>. | Understand the software development and the advantages of FPGA |
| 3. | FPGA Implementation of Naive Bayes Classifier for Visual Object Recognition | Hongying Meng, Kofi Appiah, Andrew Hunter, Patrick Dickinson<br><br>2011 | . Naïve Bayes Classifier implemented on FPGA and trained and tested using MNIST dataset.<br>The paper shows the complete design of the FPGA accelerator. It is then compared to binary self-Organizing Map (bSOM). | The hardware design of the FPGA is fully described and the accuracy of this Machine Learning implementation |
| 4. | Implementing the On-Chip Backpropagation Learning Algorithm on FPGA Architecture | Huan Minh Vo<br><br>2017 | Backpropagation learning algorithm is implemented on the FPGA. Translating the formulas to hardware circuitry is shown in detail | The results show that the error target reached 0.05 after 3 epochs the same as the Matlab. |

## 2.6 Summary

The concept of ANN is based on the human brain mechanism. The ANN is used to solve more complex and nonliner problems. The basic model of an ANN is based on synaptic interconnection, learning rule, and their activation function.

Many machine learning implementations on an FPGA illustrated the speedup performance with good efficiency.

# CHAPTER 3
# METHODOLOGY

## 3.1 Overview

This chapter presents the methodology adopted for the implementation of the proposed perceptron-based system. It begins with an outline of the overall project workflow, followed by a detailed description of the system architecture and design strategy. The implementation process is then discussed for both CPU-based and FPGA-based environments. Finally, the tools, software platforms, and design techniques utilized throughout the development are presented.

## 3.2 Flow of the project

The project is divided into two phases:

**Phase 1: Software Modeling and Algorithm Development (CPU-Based Implementation)**

1. Related work on machine learning implementations, particularly on FPGAs, studies the perceptron learning algorithm and formulates its mathematical representation.
2. Reviewing platforms.
3. Analyzing how the mathematical operations can be translated into hardware-efficient structures.
4. Developing the perceptron algorithm using an Object-Oriented Programming (OOP) approach for execution on the CPU.
5. Generating the dataset and verifying the correctness of the training process through simulation and testing.

**Phase 2: Hardware Design and FPGA Implementation**

1. Designing the hardware architecture of the perceptron system, including the Control Unit (FSM) and Datapath Unit.

2. Implementing the design using Verilog with a structural modeling approach.
3. Synthesizing and simulating the design using FPGA development tools (e.g., Vivado).
4. Evaluating system performance in terms of resource utilization, timing, and power consumption.
5. Comparing the performance results between CPU and FPGA implementations to assess speedup and efficiency.

.

## 3.3 Algorithm on CPU

### 3.3.1 The Implementation Algorithm

To describe the execution process on the CPU, the algorithm presents the procedural logic of the system. The algorithm shows how the model iteratively processes the dataset, computes prediction errors, and updates its parameters until convergence is achieved [6] [4].

**Algorithm 1** Perceptron Training on CPU

BEGIN

1: Generate dataset.

2: Initialize model parameters.

3: Train until convergence.

4: Store optimized parameters.

END

### 3.3.2 Dataset Representation and Input Scalability

The software implementation utilizes a dynamic dataset generation function to produce truth tables for fundamental logic gates, including AND, OR, NAND, and NOR. The system is designed to support a variable number of inputs $n$, providing flexibility for different classification tasks.

Scalability: The implementation supports up to 18 inputs, resulting in a dataset size of $2^{18}$ (262,144) binary combinations.

Data Structure: Each input vector is represented as a NumPy array, converting binary values into numerical features for efficient model processing [5].

High-Dimensionality: This structure allows the Perceptron to learn decision boundaries within complex logical spaces [2] [3].

### 3.3.3 Training Process and Parameter Optimization

The training process follows a stochastic learning approach, where weights and bias are updated immediately after a classification error is detected. A fixed learning rate (η) is employed to ensure stable convergence [6] [7].

In this work, a value of 0.125 was selected based on the optimization procedure described in Section 4.4.

The model is trained iteratively over multiple epochs using a loop structure that continues until the total classification error reaches zero. This indicates that the model has successfully learned the desired logic function [4].

As the number of inputs increases, the dataset size grows exponentially, resulting in higher computational complexity [2].

To analyze performance, execution time is monitored using a timing function, demonstrating the impact of input dimensionality on CPU processing time.

### 3.3.4 Weight and Bias Update Rule

The model computes output via a linear combination of inputs and weights, passed through a Heaviside step function [12]:

$$\text{Output}= \begin{cases} 1 & if\ (\sum w_i x_i + b) \geq 0 \\ 0 & Otherwise \end{cases}$$

When an error occurs, the weights ($w$) and bias ($b$) are updated using the following equations:

$$w_{new} = w_{old} + (0.125 * e * X)$$

$$b_{new} = b_{old} + (0.125 * e)$$

The selected learning rate (0.125 = ($2^{-3}$)) simplifies implementation by enabling efficient arithmetic operations.

This property supports a smoother transition to FPGA hardware, where multiplication operations can be replaced by bit-shifting techniques, reducing resource utilization and improving execution efficiency efficiency [10] [8].

## 3.4 FPGA Design and Implementation

### 3.4.1 Perceptron Training Algorithm

Before presenting the hardware implementation details, the proposed Perceptron training algorithm is described using pseudocode.

This representation outlines the sequence of operations performed during training and provides a high-level view of the algorithm independent of the underlying hardware architecture. The pseudocode serves as a reference for both the behavioral and structural implementations discussed in the following section:

**Procedure 1** FSM-Based Perceptron Training.

```
1: Wait for start signal
2: Generate dataset X and targets T
3: Initialize weights w and bias b
4: repeat
5:     Reset sample counter
6:     any_error ← 0
7:     for each sample k do
8:         Compute:
                sum = b + Σ(X[k]·w)
9:         prediction = STEP(sum)
10:        if prediction ≠ T[k] then
11:            Update bias b
```

```
12:             Update weights w
13:             any_error ← 1
14:         end if
15:     end for
16: until any_error = 0
17: Store w and b to outputs
18: done ← 1
19: return
```

### 3.4.2 Behavioral Modeling

Behavioral Verilog models describe the circuit or system at a high-level of abstraction without implying any particular structure or technology; only the overall behavior is specified. This approach is intuitive and similar to high level programming languages such as C or Python, allowing designers to focus on system functionality rather than low-level hardware components like gates and flip-flops.

In contrast to structural models which specify the exact interconnection between components at a low level of abstraction behavioral modeling enables faster simulation, easier debugging, and is technology independent, ensuring portability across different FPGA platforms [13].

In this research, the Perceptron system was initially designed using this behavioral approach to capture the complex nature of the learning algorithm. The design utilizes high-level constructs such as always blocks, if-else logic, and case statements to execute the training process. Specifically, the model describes the sequential execution of the Perceptron learning rule through the following functional steps: Accumulating the weighted sum: Iteratively calculating the dot product of inputs and weights. Output Prediction: Applying a unit step function to the accumulated sum. Error Evaluation: Comparing the prediction with the target value. Parameter Update: Adjusting weights and bias based on the learning rule. By focusing on "what the system does" rather than its gate-level implementation, this modeling style allowed for early verification of the algorithm. Modern synthesis tools, such as Vivado,

subsequently optimized this behavioral description into efficient hardware resources like Look-Up Tables (LUTs) and registers.

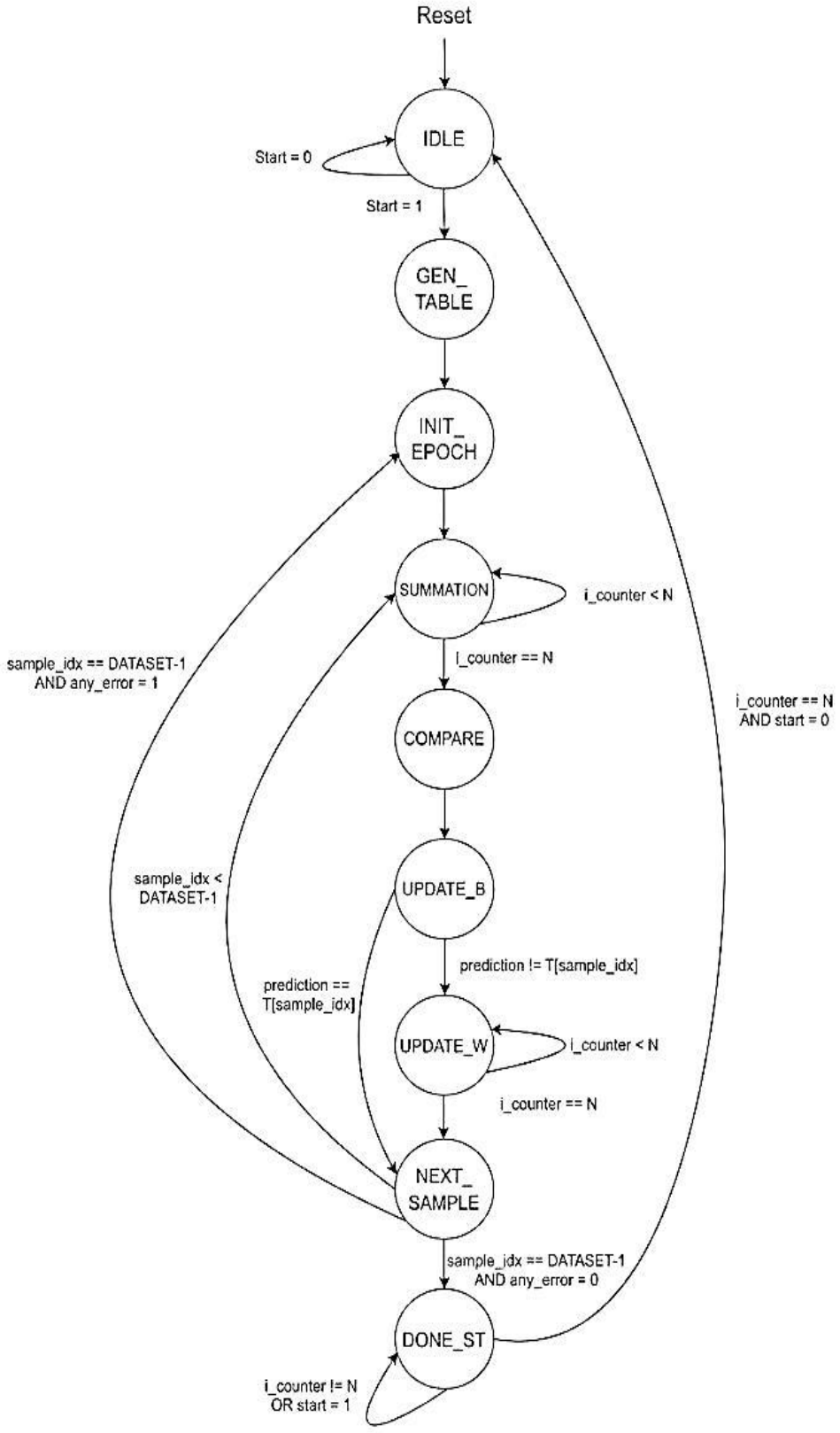



Figure 3.1 : Finite State Machine (FSM) Description for behavioral model

The proposed Perceptron system is controlled using a Finite State Machine (FSM) that organizes the training process into a sequence of states. The FSM starts in the **IDLE** state and waits for the start signal. Once activated, the training dataset is generated in the **GEN_TABLE** state, followed by the initialization of counters and training parameters in the **INIT_EPOCH** state. The system then enters the **SUMMATION** state to calculate the weighted sum of the input features. Afterward, the predicted output is determined in the **COMPARE** state and compared with the target value. If a classification error is detected, the bias and weights are updated through the **UPDATE_B** and **UPDATE_W** states according to the Perceptron learning rule. The FSM then moves to the **NEXT_SAMPLE** state to process the next training sample. This sequence is repeated until all samples in the dataset have been evaluated. At the end of each epoch, the system checks whether any classification errors occurred. If errors still exist, a new training epoch is initiated; otherwise, the FSM transitions to the **DONE_ST** state, indicating successful convergence of the training process. This FSM-based organization provides a clear and efficient control flow for the hardware implementation of the Perceptron on FPGA devices.

Although the FSM describes the sequence of state transitions, a more detailed representation is required to show the operations associated with each state. For this purpose, an ASM chart is employed to describe the complete behavior of the proposed system.

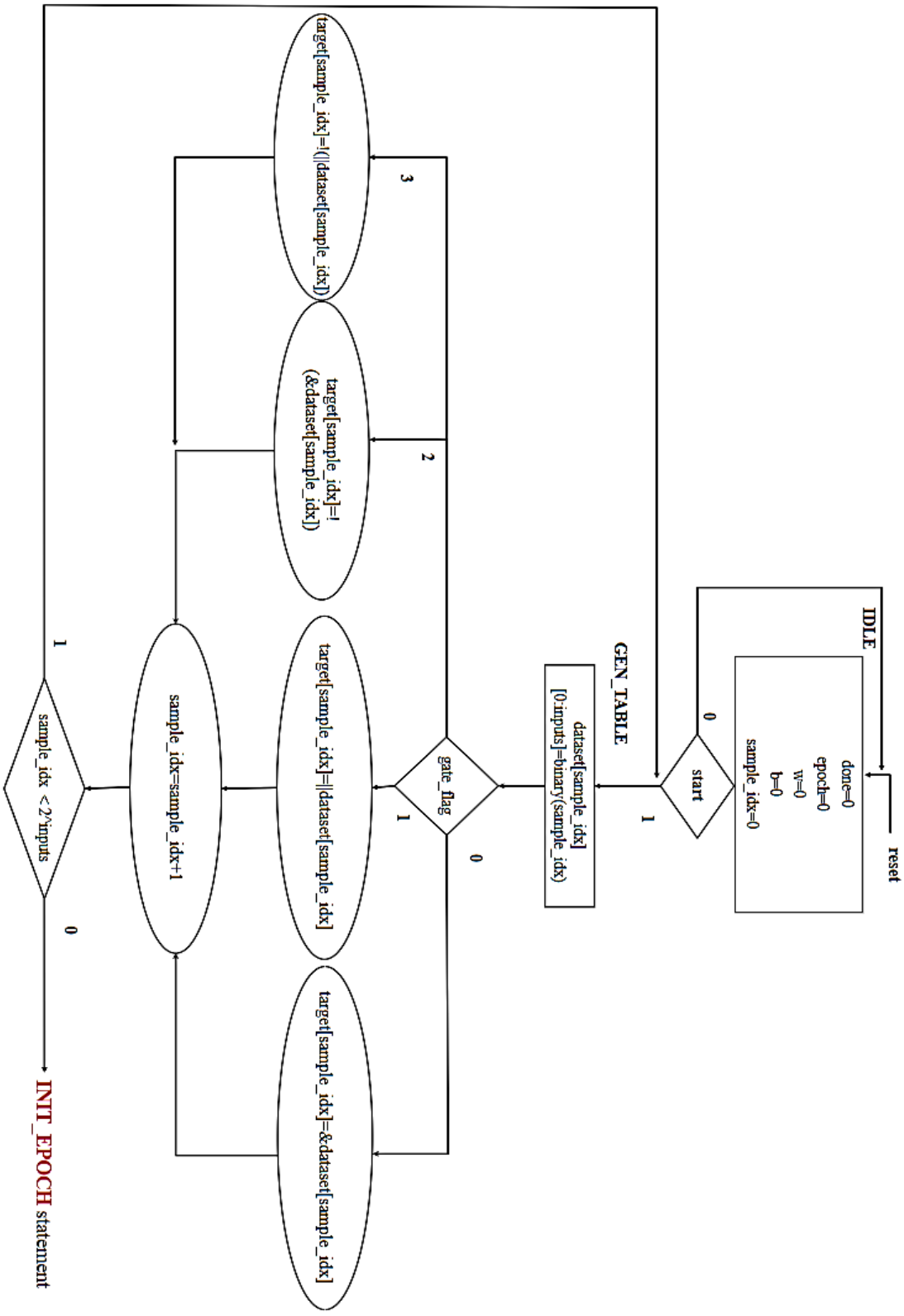



Figure 3.2: Behavioral ASM flow chart, part 1 (generation data level)

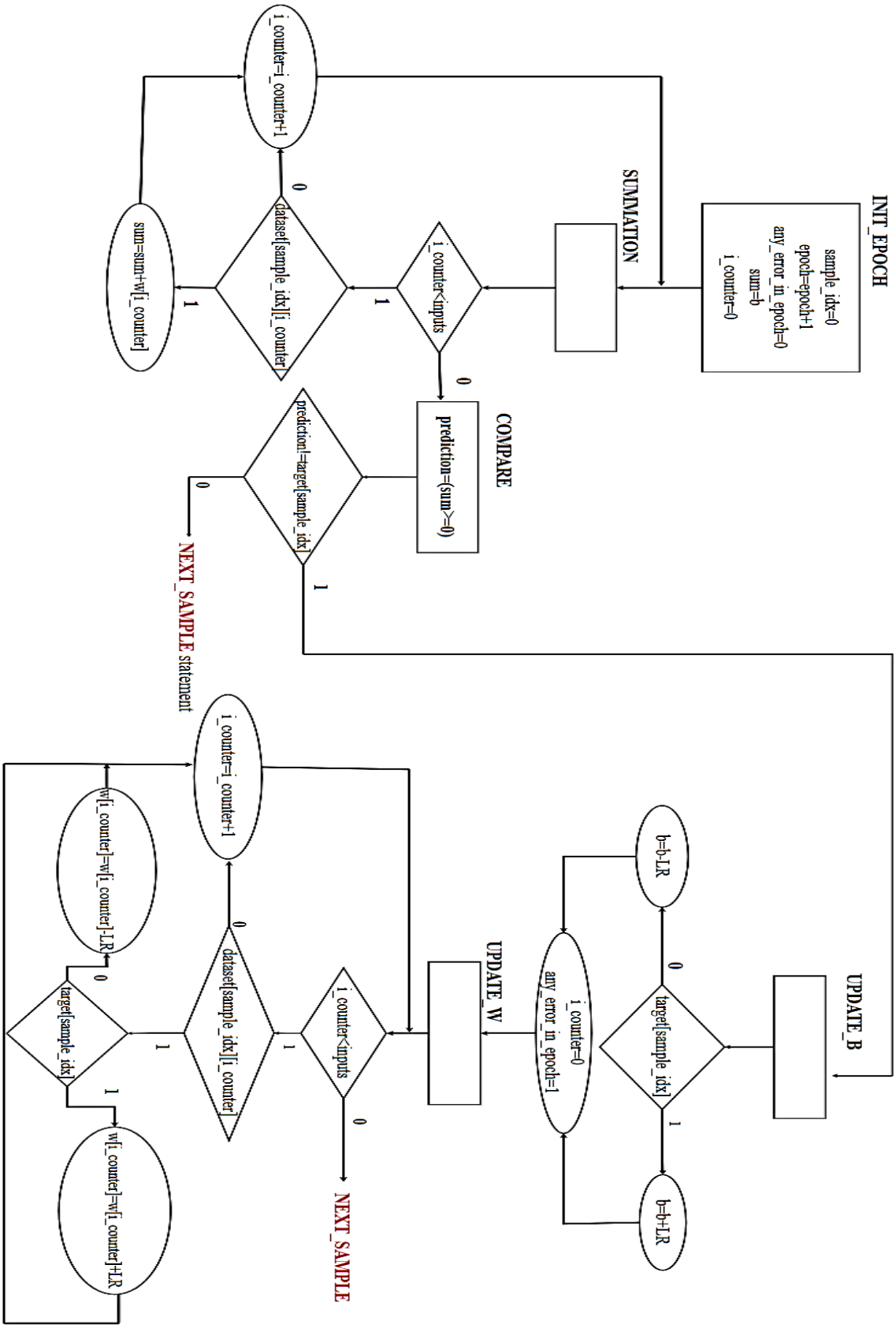



Figure 3.3: Behavioral ASM flow chart, part 2 (training and update (weights and bias) level)

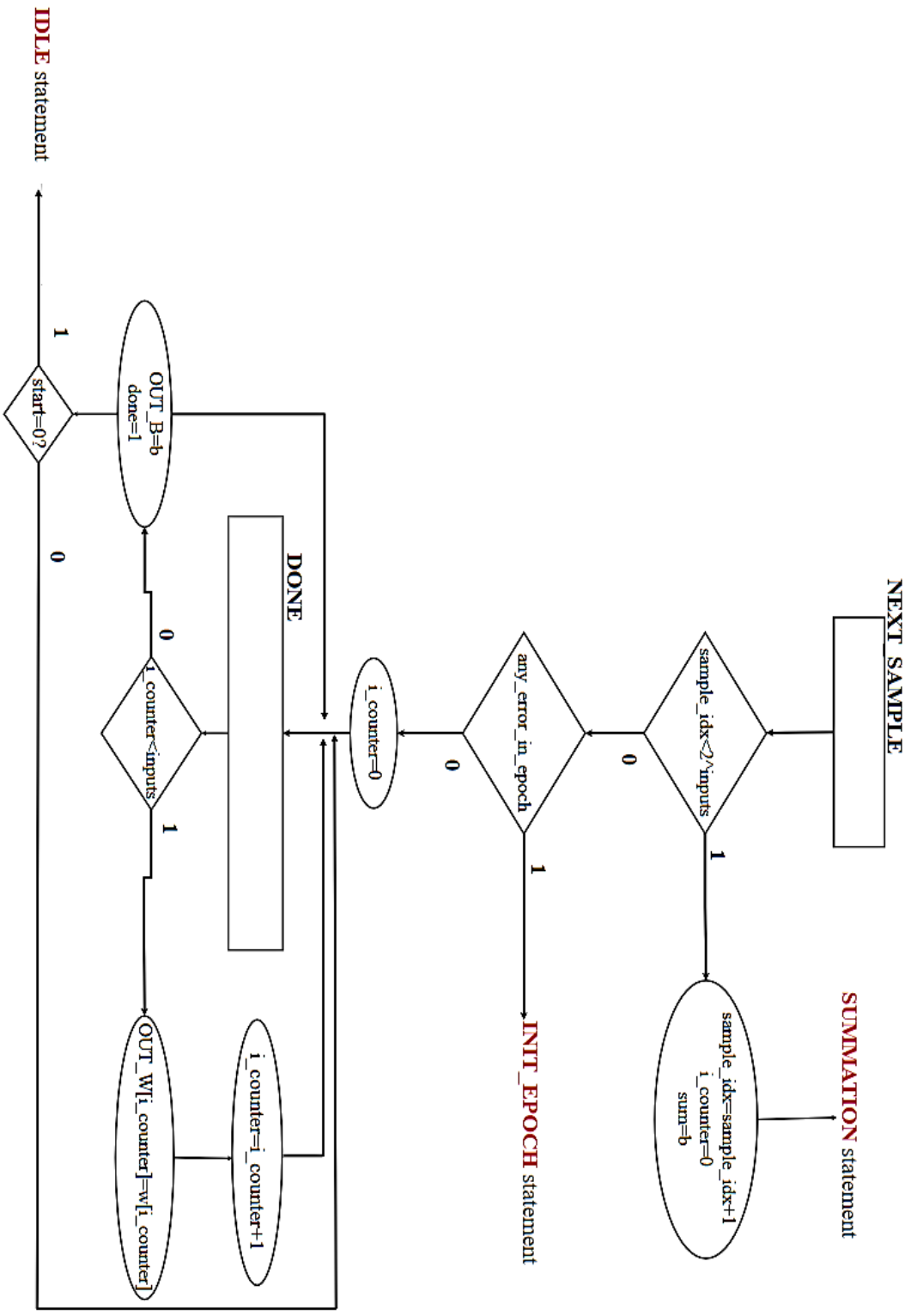



Figure 3.4: Behavioral ASM flowchart, part 3 (Dataset Traversal and Completion Check level

Initially, the system remains in the Idle state until a training request is received. Once activated, the algorithm automatically generates all possible binary input combinations and assigns the corresponding target outputs according to the selected logic gate (AND, OR, NAND, or NOR).

After dataset generation, all weights and the bias are initialized to zero. The training process then begins by starting a new epoch and resetting the sample index and error flag. For each training sample, the weighted sum is computed by accumulating the weights corresponding to active input bits and adding the bias term.

The computed sum is passed through a threshold activation function, where the output is set to logic '1' if the summation is non-negative and logic '0' otherwise. The predicted output is then compared with the target value. If a classification error is detected, the bias is updated according to the Perceptron learning rule, followed by updating the weights associated with the active input features. The update magnitude is determined by the learning rate constant.

Once processing of the current sample is completed, the algorithm advances to the next sample within the same epoch. This procedure continues until all samples in the dataset have been evaluated. At the end of each epoch, the algorithm checks whether any classification error occurred during that epoch. If at least one error is found, a new epoch is initiated, and the training process repeats. Otherwise, the network is considered converged, indicating that all samples have been correctly classified.

Finally, the learned weights and biases are transferred to the output registers, and the training completion signal is asserted to indicate successful convergence of the Perceptron model.

Despite these advantages, behavioral modeling does not explicitly represent the underlying hardware architecture, which may lead to less optimal resource utilization. Therefore, it is mainly used as an initial design step before transitioning to a more hardware-oriented structural implementation.

### 3.4.3 Structural Modeling and Hardware Architecture

To achieve optimal execution speed, high hardware efficiency, and clear modularity, the Perceptron system was transitioned from a high-level behavioral description to a hardware-optimized structural implementation [13].

This structural approach explicitly partitions the neural network processor into two major subsystems: a Control Unit (CU) and a Datapath Unit (DU), operating in a synchronized manner.

By separating the control logic from the arithmetic processing hardware, the design achieves deterministic timing behavior, improves scalability, simplifies debugging, and facilitates FPGA implementation [8,13].

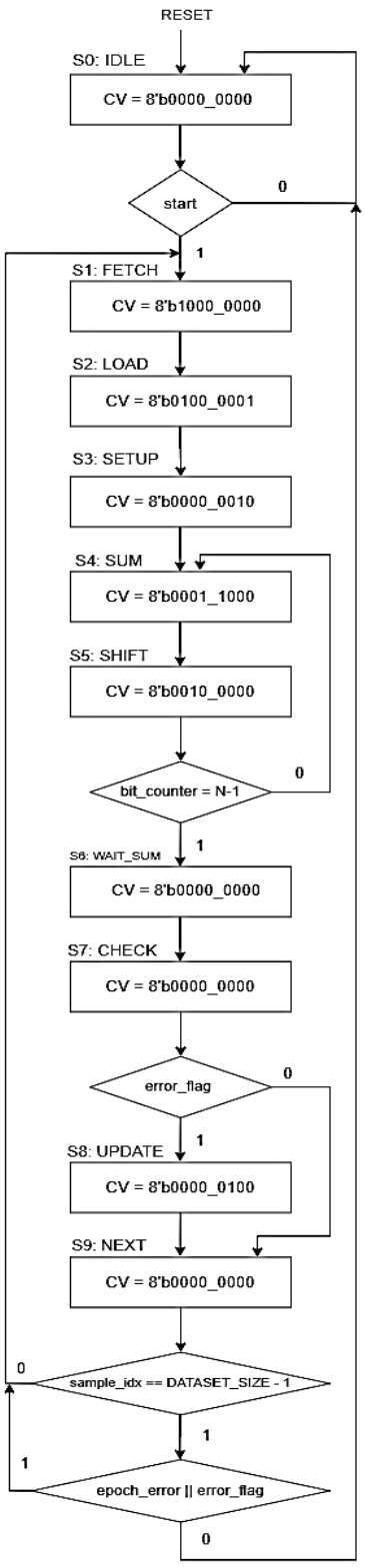



Figure 3.5: Algorithmic State Machine (ASM) chart describing the sequential execution flow and state transitions of the hardware Perceptron trainer.

#### 3.4.3.1 Control Unit (CU) and Finite State Machine (FSM)

The Control Unit (CU) serves as the central supervisory entity within the hardware architecture, orchestrating the complete training workflow of the perceptron system. It is implemented as a synchronous Finite State Machine (FSM), enabling deterministic and cycle-accurate control over all computational and data movement operations. The CU continuously monitors internal status signals and dynamically generates a control vector (CV), which governs the activation and coordination of functional components within the Datapath Unit (DU) [13].

The operation of the FSM is structured into a well-defined sequence of states, ensuring an efficient mapping of the perceptron learning algorithm onto hardware. The process begins in **S0 (IDLE)**, where the system remains in a standby condition awaiting an external start signal. Upon activation, the FSM transitions to **S1 (FETCH)**, in which the required training sample is retrieved from LUT-based memory, followed by **S2 (LOAD)**, where both the input vector and corresponding target value are stored in internal registers for processing.

The computational phase is initiated in **S3 (SETUP)**, where the bias value is loaded into the accumulator to initialize the summation process. Subsequently, in **S4 (SUM)**, the weighted accumulation is performed iteratively, processing one input bit per clock cycle. This sequential approach minimizes hardware resource utilization while maintaining computational correctness.

The **S5 (SHIFT)** state advances the input shift register and updates the bit counter, enabling systematic traversal of the input vector. To ensure reliable timing behavior, the FSM enters **S6 (WAIT_SUM)**, a dedicated synchronization state that guarantees stabilization of the accumulated result before evaluation.

Decision-making is carried out in **S7 (CHECK)**, where the activation logic—implemented as a sign-based comparator—generates the predicted output and compares it against the target value to detect classification errors. In the presence of an error, the FSM transitions to **S8 (UPDATE)**, where the Datapath Unit performs weight and bias adjustments according to the perceptron learning rule. Notably, due to the fixed learning rate of 0.125, these

updates are efficiently realized using shift-based arithmetic operations, eliminating the need for hardware multipliers and reducing resource consumption [8,10].

Finally, in **S9 (NEXT)**, the Control Unit determines whether to proceed to the next training sample or initiate a new training epoch. This tight integration between the Control Unit and the Datapath Unit ensures a fully synchronized execution model, enabling efficient hardware realization of iterative learning while optimizing latency, resource utilization, and scalability.

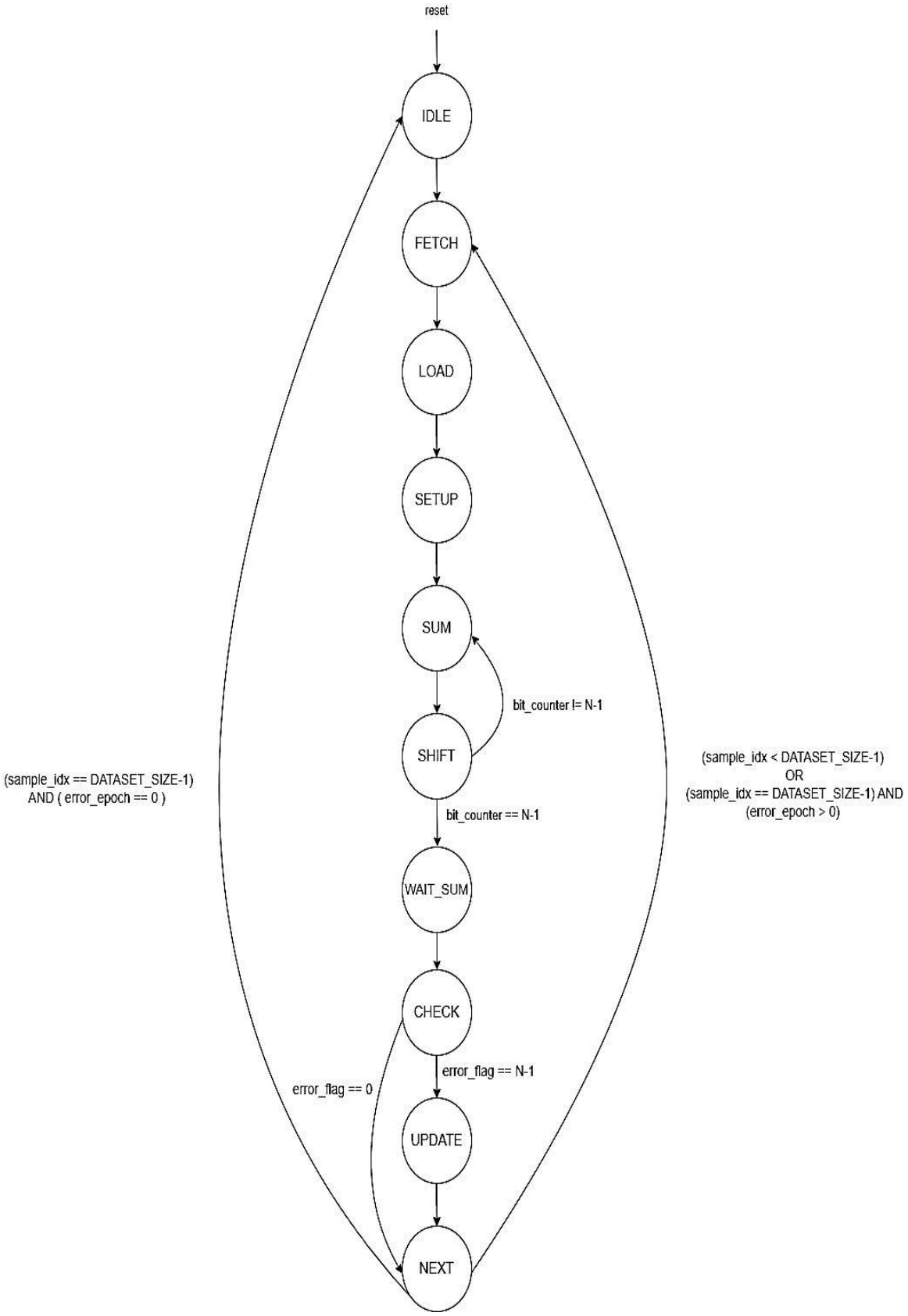



Figure 3.6: FSM state transition and transition conditions.

Table 3.1: RTL-CS Table for Structural Perceptron Trainer

| State | RTL Code | Asserted Control Signals | CV[7:0] |
|---|---|---|---|
| S0: IDLE | (start ∧ ¬done) / state ← FETCH | None | 00000000 |
| S1: FETCH | {vector_x,target} ← M[sample_idx] | enabl_lut | 10000000 |
| S2: LOAD | shift_reg ← vector_x, x_hold ← vector_x, target_reg ← target, sum_reg ← 0 | enable_sample, sum_reg_reset | 01000001 |
| S3: SETUP | sum_reg ← bias_reg | load_bias_to_sum | 00000010 |
| S4: SUM | (shift_reg[0]) / sum_reg ← sum_reg + weight_reg[bit_counter] | enable_sum, select_x | 00011000 |
| S5: SHIFT | shift_reg ← shift_reg >> 1 | enabl_shift | 00100000 |
| S6: WAIT_SUM | No RTL operation | None | 00000000 |
| S7: CHECK | (sum_reg ≥ 0) → error_flag | None | 00000000 |
| S8: UPDATE | (target_reg) / {bias_reg,weight_reg[i]} ← {bias_reg,weight_reg[i]} + LR (¬target_reg) / {bias_reg,weight_reg[i]} ← {bias_reg,weight_reg[i]} - LR | select_bw | 00000100 |
| S9: NEXT | (sample_idx < DATASET_SIZE-1) / sample_idx ← sample_idx + 1 (sample_idx = DATASET_SIZE-1 ∧ ¬epoch_error ∧ ¬error_flag) / done ← 1 (sample_idx = DATASET_SIZE-1 ∧ (epoch_error ∨ error_flag)) / epoch ← epoch + 1, sample_idx ← 0 | None | 00000000 |

Control vectors (CV) are a concatenation of :

enabl_lut & enable_sample & enabl_shift & enable_sum & select_x & select_bw & load_bias_to_sum & sum_reg_reset.

### 3.4.3.2 Datapath Unit (DU)

The Datapath Unit (DU) represents the computational core of the hardware Perceptron implementation. It contains the arithmetic and storage elements responsible for executing the learning algorithm under the supervision of the Control Unit [13]. The main components of the Datapath Unit include:

- **Weight and Bias Registers**

  These registers store the trainable parameters using a signed 32-bit fixed-point representation (Q6.26 format) [8,13].

- **Accumulator**

  This unit performs the weighted summation required for the Perceptron decision function [13].

- **Activation Logic**

  The activation function is implemented as a step function by evaluating the sign of the accumulated sum [6].

- **Weight Update Logic**

  This block updates the weights and bias according to the Perceptron learning rule whenever a classification error occurs [6,12].

Due to the fixed learning rate ($\eta = 0.125$), the update operation is implemented using shift-based arithmetic instead of hardware multipliers. This design choice significantly reduces FPGA resource utilization while improving computational efficiency [8,10].

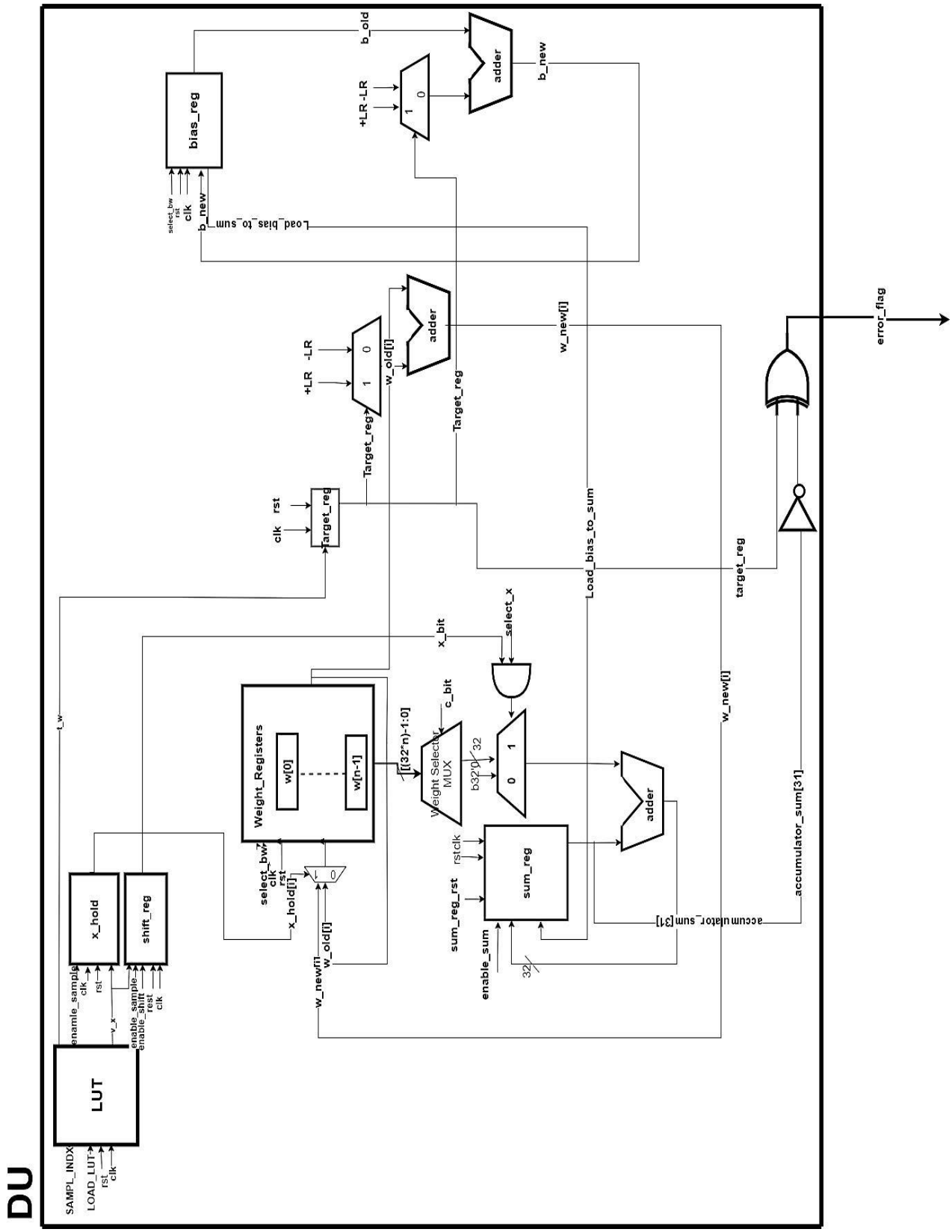



Figure 3.7: Detailed Block Diagram of the Datapath Unit (DU).

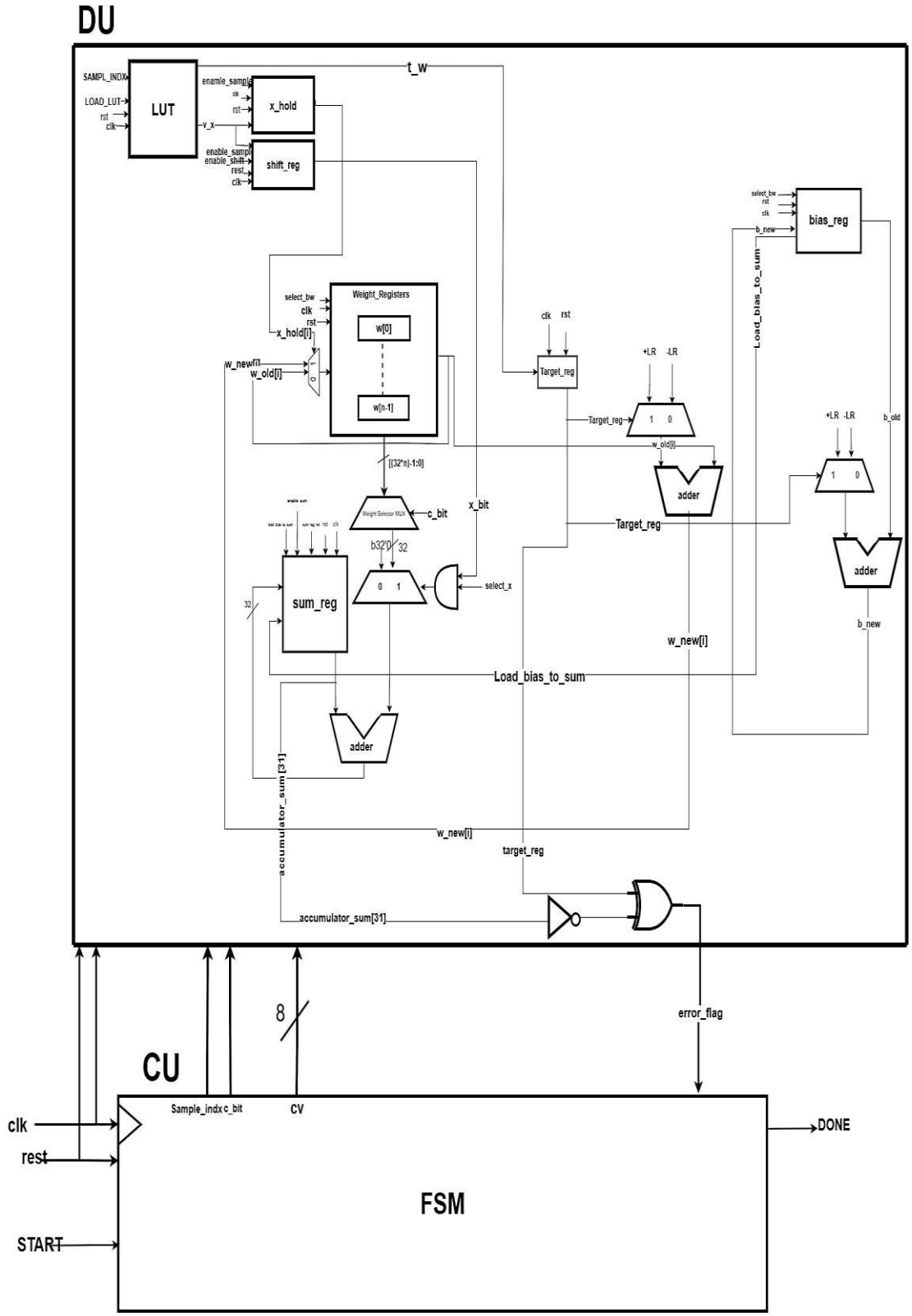



Figure 3.8: Structural hardware architecture showing the interaction between the control Unit and the Datapath Unit.

#### 3.4.3.3 Timing Synchronization Between CU and DU

During post-implementation timing analysis, a synchronization issue was observed between the Control Unit and the Datapath Unit.

Although the design operated correctly during behavioral simulation, timing delays introduced by FPGA routing and logic propagation caused the error evaluation stage to occasionally read an unstable accumulated sum value. To eliminate this timing mismatch, an additional synchronization state (WAIT_SUM) was inserted between the summation phase and the classification phase. This state provides one extra clock cycle for the accumulator output to settle completely before the error comparison logic is activated. As a result, stable communication between the CU and DU was guaranteed under post-implementation timing conditions, ensuring identical functionality in both simulation and synthesized hardware execution.

### 3.4.4 LUT- based design

A Look-Up Table (LUT) is a fundamental building block in FPGA architectures used to implement combinational logic. It stores the truth table of a logic function and produces outputs based on the applied inputs, eliminating the need for explicit gate-level implementations [13].

Although LUTs are not directly instantiated in the Verilog code, they represent the primary hardware resource utilized during synthesis. The synthesis tool automatically maps behavioral descriptions, including arithmetic operations, conditional statements, and finite state machine (FSM) transitions, into LUT-based implementations [13].

In the proposed Perceptron system, LUTs are mainly used to implement control logic, decision-making circuits, and parts of the activation function. The activation function is realized as a comparator that determines whether the computed sum is greater than or equal to zero using the Most Significant Bit (MSB), also known as the sign bit in fixed-point representation.

Additionally, due to the binary nature of the input dataset, multiplication operations are significantly simplified during synthesis. In many cases, these

operations are reduced to simple shift-and-add or routing logic, which can be efficiently mapped into LUT-based resources instead of consuming dedicated DSP blocks.

This LUT-based implementation improves resource utilization and scalability while maintaining efficient performance. As a result, the design is well-suited for low-cost FPGA devices with limited arithmetic resources. The effectiveness of this approach is further demonstrated through the resource utilization and power analysis results presented in Chapter 4.

## 3.5 Fixed point representation

Fixed-point representation is a numerical format in which the binary point is placed at a fixed position within the number. The total number of bits is divided between the integer part and the fractional part, and the position of the binary point determines both the range and precision of the representation.

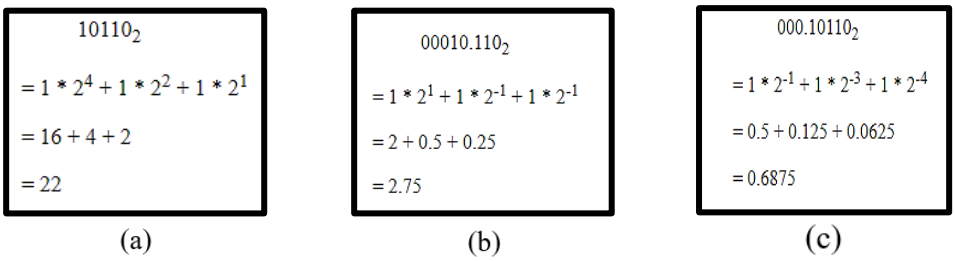



Figure 3.9: Idea Of Fixed-Point Numbers

In Figure 3.9 (a), the binary representation for integer 22. If the floating point is added after the third bit, as shown in Figure 3.9 (b), the number will be 2.75. If the floating point is added after the fifth bit, the number will be 0.6875 as shown in Figure 3.9 (c). The mathematical principle is that the bits on the left of the point will be considered as 2 to the power of a positive number. And the bits on the right of the point will be considered as 2 to the power of a negative number.

### 3.5.1 Motivation of the fixed point

The implementation of neural networks on an FPGA requires an efficient numerical representation. Although floating-point arithmetic provides high precision and a wide dynamic range, it is not well-suited for FPGA-based designs due to its high hardware complexity.

Therefore, fixed-point representation is adopted in this work for several key reasons:

- **Hardware Efficiency:** Fixed-point operations rely on simple integer arithmetic, which significantly reduces the utilization of logic resources such as LUTs and registers.
- **Higher Speed and Lower Latency**: Fixed-point computation avoids normalization and exponent handling, resulting in faster execution and reduced latency.
- **Reduced Hardware Cost:** Unlike floating-point units that often require dedicated DSP resources, fixed-point operations can be efficiently implemented using basic FPGA logic.
- **Adequate Precision for the Application:** The perceptron model used for logic gate learning does not require high numerical precision, making fixed-point representation sufficient for achieving convergence.

Based on these advantages, a fixed-point format is selected to achieve a balance between hardware efficiency and computational accuracy. This design choice is validated by the implementation results presented in Chapter 4.

### 3.5.2 Representation of Weights and Bias

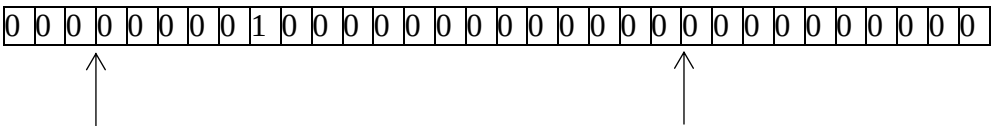
In this design, the weights and bias of the perceptron are represented using a fixed-point format to ensure efficient hardware implementation. The selected format divides the total number of bits into integer and fractional parts, allowing a balance between numerical range and accuracy. Specifically, a fixed-point representation of (6 integer bits and 26 fractional bits) is adopted. This format provides sufficient dynamic range to represent both positive and negative values, while maintaining high precision for small fractional updates

during the training process. The weights and biases are stored as signed values using two's complement representation, which enables efficient implementation of arithmetic operations such as addition and subtraction in hardware. This representation is particularly important for the weight update process, where small incremental changes must be accurately captured to ensure proper convergence of the perceptron.

For example :

When training the perceptron on 3 inputs, the target logic gate in the training is also an AND GATE, the result of this operations in binary :

**1) $W_0$ =**




**Integer part (6bits)** **Fraction part(26bits)**

$W_0$= 000000 | 00100000000000000000000000

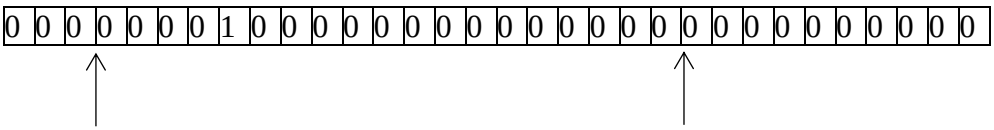
When converting from binary to decimal systems

Value = $2^{23}$ = 8388608

$W_0 = \frac{8388608}{2^{26}}$ = 0.125

**2) By using the same way for $W_1$,$W_1$ = 0.125**

**3) $W_2$ =**



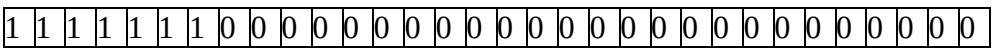

**Integer part (6bits)** **Fraction part(26bits)**

$W_2$= 000000 | 01000000000000000000000000

$W_2 = 2^{24}$ = 16777216

$W_2 = \frac{16777216}{2^{26}}$ = 0.25

**4) Bias =**



This value is negative since the most significant bit (MSB) is equal to 1; we apply 2`s complement first :

| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

We add +1 :

| 0 | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

$2^{25}$ = -33554432

Bias = - $\frac{33554432}{2^{26}}$ = -0.5

Or use another method :

$$-( 2^{31} - 2^{30} - 2^{29} - 2^{28} - 2^{27} - 2^{26} - 2^{25} ) = -33554432$$

Bias = $\frac{-33554432}{2^{26}}$ = -0.5

Table 3.2: Fixed-Point Representation Examples

| Binary (32-bit) | Hex | Integer | Fixed-Point Fraction | Actual |
|---|---|---|---|---|
| 00000010000000000000000000000000 | 02000000 | 33554432 | 0.5000000000 | 1/2 |
| 00000001000000000000000000000000 | 01000000 | 16777216 | 0.2500000000 | 1/4 |
| 00000000100000000000000000000000 | 00800000 | 8388608 | 0.1250000000 | 1/8 |
| 00000000010000000000000000000000 | 00400000 | 4194304 | 0.0625000000 | 1/16 |
| 00000000001000000000000000000000 | 00200000 | 2097152 | 0.0312500000 | 1/32 |
| 00000000000100000000000000000000 | 00100000 | 1048576 | 0.0156250000 | 1/64 |
| 00000000000010000000000000000000 | 00080000 | 524288 | 0.0078125000 | 1/128 |
| 00000000000001000000000000000000 | 00040000 | 262144 | 0.0039062500 | 1/256 |
| 00000000000000100000000000000000 | 00020000 | 131072 | 0.0019531250 | 1/512 |
| 00000000000000010000000000000000 | 00010000 | 65536 | 0.0009765625 | 1/1024 |
| 00000000000000001000000000000000 | 00008000 | 32768 | 0.0004882813 | 1/2048 |
| 00000000000000000100000000000000 | 00004000 | 16384 | 0.0002441406 | 1/4096 |
| 00000000000000000010000000000000 | 00002000 | 8192 | 0.0001220703 | 1/8192 |
| 00000000000000000001000000000000 | 00001000 | 4096 | 0.0000610352 | 1/16384 |

### 3.5.3 Range of Q6.26 Fixed-Point Format

The Q6.26 fixed-point representation provides a numerical range from -32 to approximately +31.999999985, offering sufficient precision and dynamic range for the perceptron training and inference processes.

Table 3.3: Range of Q6.26 Fixed-Point Representation

| Type | Binary Representation | Fixed-Point Value |
|---|---|---|
| Maximum Positive Value | 01111111111111111111111111111111 | +31.999999985 |
| Minimum Negative Value | 10000000000000000000000000000000 | -32.000000000 |

### 3.5.4 Hardware efficiency impact

The choice of fixed-point representation has a significant impact on the hardware efficiency of the implemented perceptron system. Compared to floating-point arithmetic, fixed-point operations are simpler and require fewer hardware resources, making them more suitable for FPGA-based designs. In terms of resource utilization, the use of fixed-point arithmetic reduces the complexity of the arithmetic units. This may lead to a reduction in Look-Up Tables (LUTs) and Flip-Flops (FFs), depending on the synthesis optimization and design architecture. Fixed-point operations rely on integer-based computations, eliminating the need for dedicated floating-point units, which are typically resource-intensive. Moreover, fixed-point arithmetic simplifies the computational pipeline by avoiding operations such as exponent alignment and normalization required in floating-point representations. This generally results in reduced latency and more efficient hardware mapping, which can improve overall system performance in real-time applications. In addition, the design reduces dependency on dedicated DSP blocks. Due to the simplicity of the implemented operations and the binary nature of the input dataset, several arithmetic functions can be mapped onto LUT-based logic instead of DSP

resources. This enhances the portability of the design, especially for low-cost FPGA devices with limited DSP availability. From a power consumption perspective, the implementation demonstrates efficient energy usage. The total power consumption, obtained from post-implementation synthesis and power analysis reports, remains relatively low. This reduction is mainly attributed to the simplified arithmetic operations and reduced switching activity associated with fixed-point representation. The dynamic power component is influenced by the number of active transitions in the design, while static power depends primarily on the target FPGA technology. Overall, the use of fixed-point representation achieves a balanced trade-off between hardware efficiency, computational performance, and power consumption, making it a suitable choice for the perceptron implementation in this work.

# CHAPTER 4
# RESULTS

## 4.1 Overview

This chapter provides a comprehensive analysis of the results obtained from both the software simulation and hardware implementation of the Perceptron model. The primary objective is to evaluate the model's accuracy, convergence behavior, and computational efficiency across varying input dimensions and hyperparameter configurations. The transition from CPU-based processing to FPGA acceleration is also analyzed to justify the hardware-centric approach for high-dimensional logical operations.

## 4.2 CPU-Based Implementation and Procedural Logic

While the theoretical foundation was established in the previous chapter, this section focuses on the technical realization of the model on the Central Processing Unit (CPU). The implementation adopts an Object-Oriented Programming (OOP) approach to translate the mathematical update rules into a structured, executable format. This architectural choice ensures that the CPU can efficiently manage memory and process the dynamic scaling of datasets through a modular design [11].

### 4.2.1 Modular Organization

To optimize the CPU's execution flow, the system is partitioned into technical classes that encapsulate the training logic [11]:

- **Dataset Handler**: A specialized class responsible for the automated construction of truth tables and logic gate mapping based on the defined input scale.

- **Perceptron Engine:** A dedicated class that encapsulates the model's essential parameters, including weight vectors, bias constants, and the core training functions executed by the CPU.

### 4.2.2 Implementation Verification (Case Study: n = 6)

To validate the practical execution of the aforementioned algorithm, a simulation was conducted for an AND gate configuration with 6 inputs (n=6). The following screenshot captures the CPU output, demonstrating the initialization of weights, the iterative error reduction process, and the final convergence where the total error reaches zero.

```
Converged at epoch 6
------------------------------
Final weights: [0.25  0.125]
Final bias: -0.375
Execution time: 0.005803 seconds
Accuracy: 100.00%
```

(a) Inputs = 2

```
Converged at epoch 16
------------------------------
Final weights: [0.5   0.25  0.125 0.125]
Final bias: -1.0
Execution time: 0.004071 seconds
Accuracy: 100.00%
```

(b) Inputs = 4

```
Converged at epoch 56
--------------------------
Final weights: [0.75  0.625 0.5   0.25  0.125 0.125]
Final bias: -2.375
Execution time: 0.073000 seconds
Accuracy: 100.00%
```

(c) Inputs = 6

```
Converged at epoch 77
------------------------------
Final weights: [1.    0.75  0.625 0.5   0.125 0.125 0.125 0.125]
Final bias: -3.375
Execution time: 0.100931 seconds
Accuracy: 100.00%
```

(d) Inputs = 8

Figure 4.1: CPU Training Log and Output for a 6-input AND Gate Configuration.

## 4.3 Convergence Analysis

The convergence behavior was visualized to monitor the reduction of the total error during the training process [4]. Figure 4.2 illustrates the stability of the model as the number of epochs increases, particularly for higher-dimensional datasets.

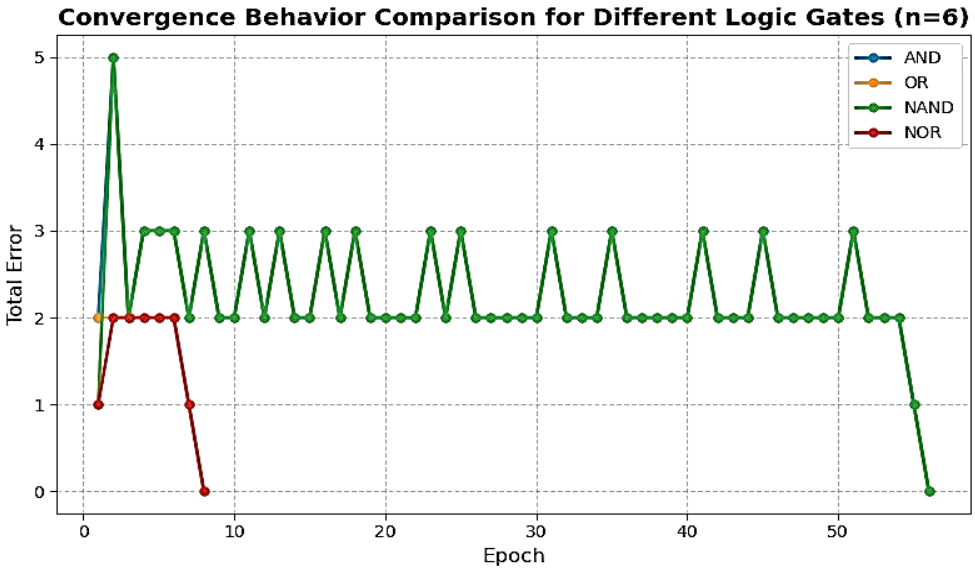



Figure 4.2: Convergence behavior comparison for different logic gates at $n$=6.

It is observed that AND and NAND gates exhibit higher oscillation and require more epochs (56) compared to OR and NOR gates.

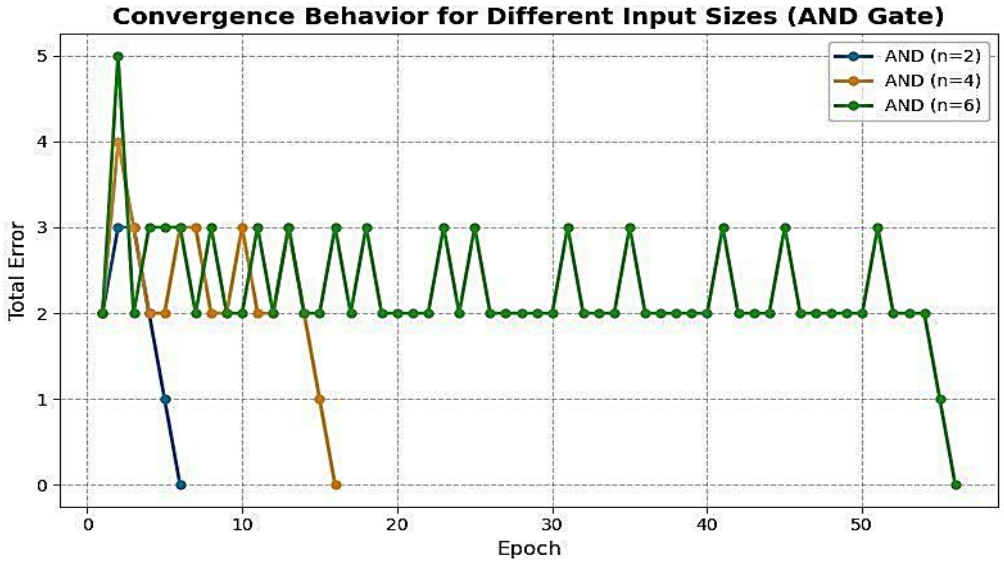



Figure 4.3: Impact of scaling input dimensions on AND gate convergence

The results indicate a direct correlation between input dimensionality and the number of iterations required for the model to achieve zero error [2] [4].

## 4.4 Learning Rate Optimization and Sensitivity Analysis

To ensure the robustness of the Perceptron model, a systematic sensitivity analysis was conducted to identify the optimal learning rate ($\eta$) [3] [4].

This optimization process followed a structured testing scenario, where various rates were evaluated using a fixed 5-input configuration as a benchmark. By monitoring the number of epochs required for convergence across different logic gates, a data-driven basis was established for selecting the final parameter.

**Algorithm 2** Learning Rate Selection

BEGIN

1: Generate the dataset.

2: Test different learning rate values.

3: Evaluate convergence performance.

4: Select the optimal learning rate.

5: Use the selected value for FPGA implementation.

END

The empirical data gathered from this testing scenario are summarized in Table 4.1. The results demonstrate that while multiple learning rates lead to successful convergence, the selection of 0.125 was a strategic decision based on the trade-off between software performance and hardware constraints. As a power of two ($2^{-3}$), this value is critical for the subsequent FPGA implementation. It allows the system to replace resource-heavy multipliers with efficient Arithmetic Right-Shift operations, thereby optimizing power consumption and logic area without compromising the model's accuracy [8].

Table 4.1: Impact of Learning Rate on Convergence (n = 6)

| **Learning rate ($\eta$)** | **AND** | **OR** | **NAND** | **NOR** |
|---|---|---|---|---|
| 0.100 | 20 | 7 | 16 | 7 |
| 0.110 | 21 | 7 | 33 | 7 |
| 0.125 (selected) | 18 | 7 | 18 | 7 |
| 0.130 | 20 | 7 | 20 | 7 |
| 0.150 | 32 | 7 | 18 | 7 |
| 0.180 | 20 | 7 | 20 | 7 |

Justification for Selecting $\boldsymbol{\eta = 0.125}$: The selection of 0.125 ($2^{-3}$) was driven by hardware constraints. In FPGA implementation, this value allows the system to replace resource-intensive multipliers with simple arithmetic right-shift operations. This optimization significantly reduces the power consumption and logic area of the design while maintaining rapid convergence [8] [10].

## 4.5 FPGA Implementation and Hardware Results

### 4.5.1 Behavioral model result and analysis

#### 4.5.1.1 Run Post-Implementation Timing Simulation

Post-Implementation Timing Simulation is performed after synthesis and implementation to verify the functionality of the FPGA design under realistic timing conditions. Unlike behavioral simulation, this stage includes propagation and routing delays introduced during placement and routing inside the target FPGA device.

The simulation utilizes the implemented hardware netlist generated after the routing process, allowing the timing behavior of the design to be evaluated more accurately. Consequently, the obtained waveforms reflect both logical correctness and timing-related hardware effects, including signal propagation delays and clock distribution delays.

In this study, post-implementation timing simulation was used to validate the operation of the perceptron system and analyze the timing behavior of control and output signals after FPGA implementation. The generated timing results provide a close estimation of the expected execution behavior on the target FPGA hardware prior to physical deployment. Further simulation results for different number of inputs using AND GATE:

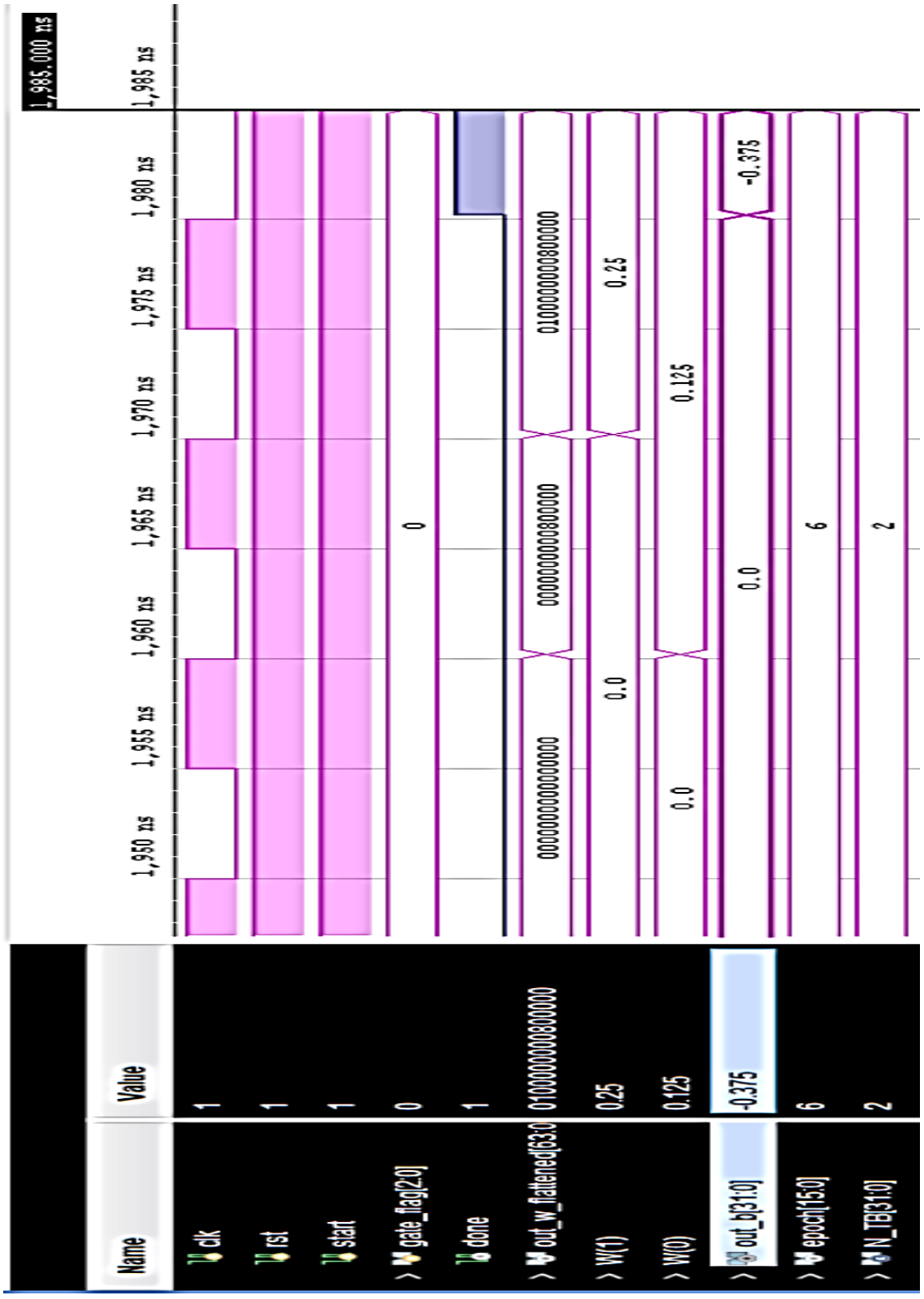



Figure 4.4 : results for 2- inputs behavioral design.

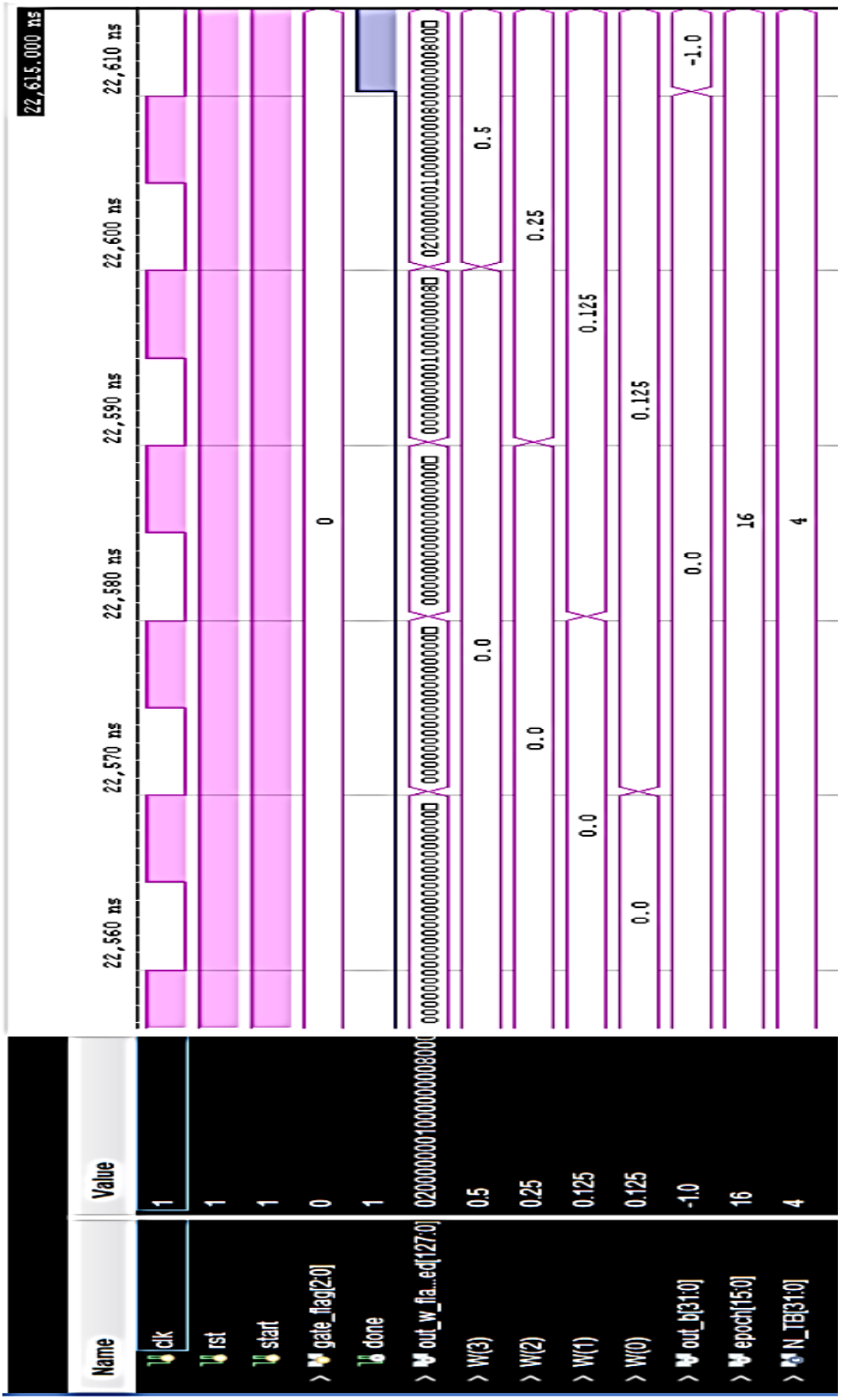



Figure 4.5 : results for 4- inputs behavioral design.

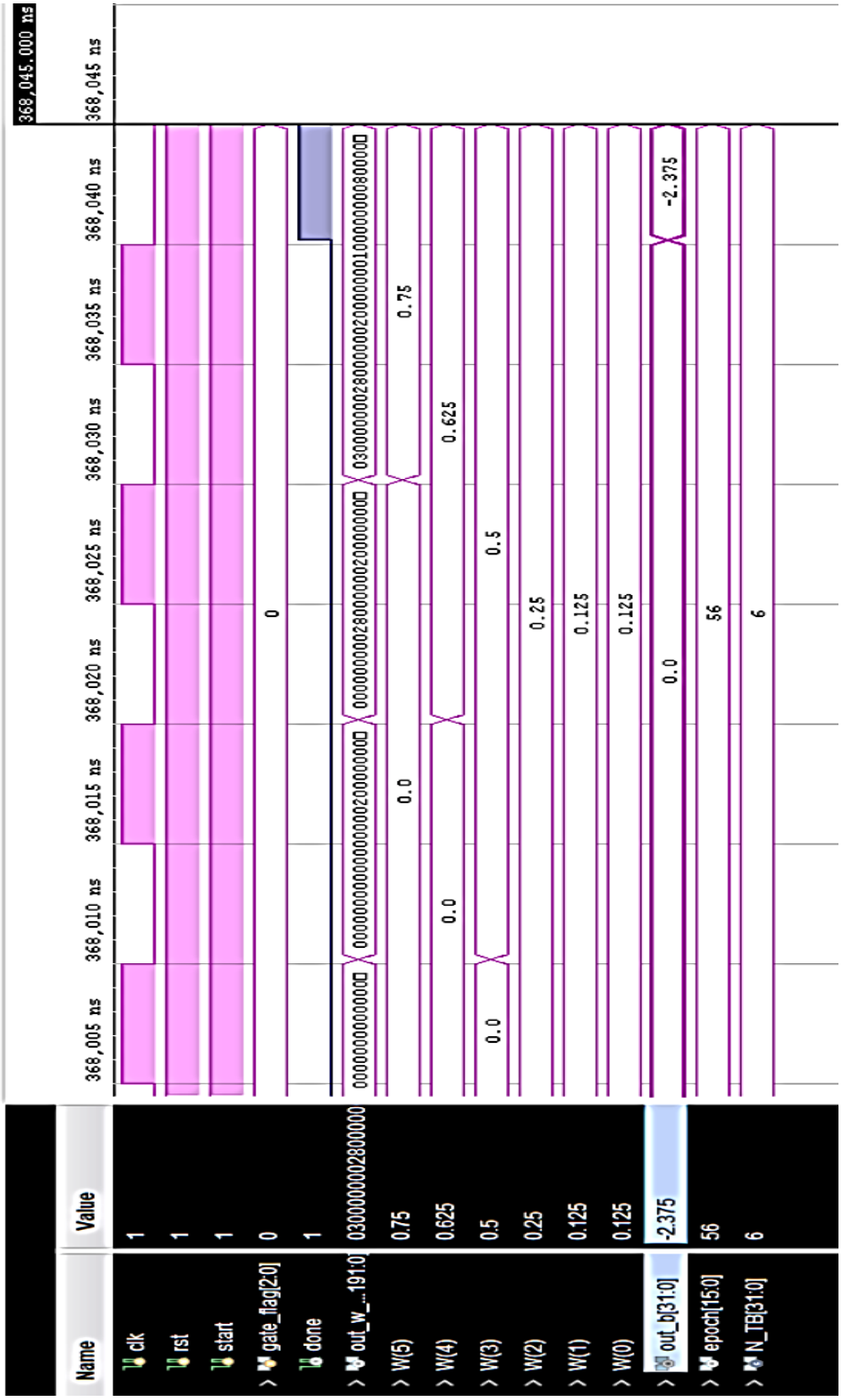



Figure 4.6 : results for 6- inputs behavioral design.

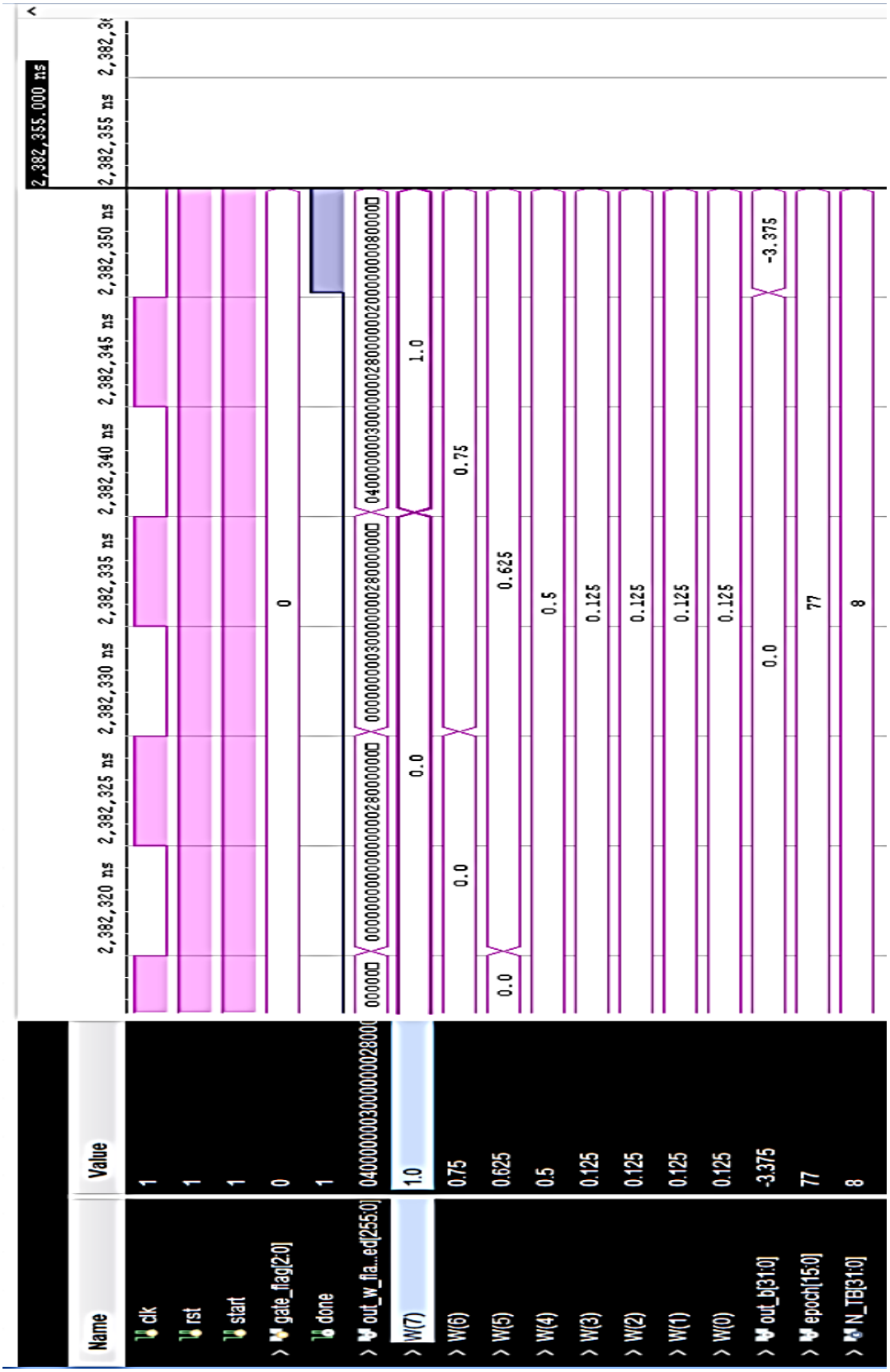



Figure 4.7 : results for 8- inputs behavioral design.

The waveform shown in Figure 4.6 presents the results obtained from the Post-Implementation Timing Simulation of the proposed perceptron system after FPGA synthesis and implementation. This simulation was performed to verify the functional correctness of the hardware design under realistic timing conditions, including propagation and routing delays introduced during the implementation stage.

The displayed signals confirm the correct operation of the training process and control logic of the perceptron architecture. The clock (clk) signal drives the sequential operation of the system, while the reset (rst) and start (start) signals initialize and activate the training process. The done signal indicates the successful completion of the execution process after all training iterations have been finalized.

The waveform also illustrates the evolution of the perceptron weight values (W (5) to W (0)), which are represented using fixed-point arithmetic.

These values change progressively during the training procedure according to the implemented learning algorithm. The observed transitions demonstrate that the weight update mechanism operates correctly within the implemented hardware architecture.

#### 4.5.1.2 Behavioral Resource Utilization and Power Analysis

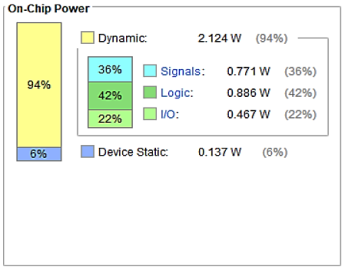
The physical implementation footprint and power efficiency of the proposed perceptron system were evaluated across different configurations (2, 4, and 6 inputs) on the target FPGA device. The consolidated results extracted from the Vivado post-implementation reports are summarized in Table 4.2.

Table 4.2: Resource Utilization and Power Consumption vs. Number of Inputs.

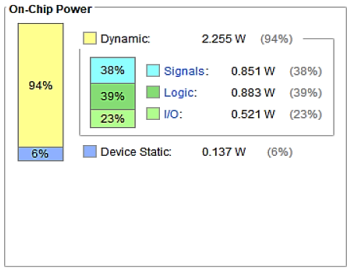
| **Inputs** | **LUTs** | | **Flip-flops** | | **DSP** | **I/O** | | **BUFGs** | | **Total-power (w)** |
|---|---|---|---|---|---|---|---|---|---|---|
| 2 | **used** | **total** | **used** | **total** | 0 | **used** | **total** | **used** | **total** | 2.261 |
| | 188 | 133800 | 183 | 267600 | | 120 | 400 | 1 | 32 | |

| Inputs | LUTs | | Flip-flops | | DSP | I/O | | BUFGs | | Total-power (w) |
|---|---|---|---|---|---|---|---|---|---|---|
| 4 | used | total | used | total | 0 | used | total | used | total | 2.392 |
| | 223 | 133800 | 223 | 267600 | | 184 | 400 | 1 | 32 | |
| 6 | 236 | 133800 | 263 | 267600 | 0 | 248 | 400 | 1 | 32 | 2.795 |
| 8 | 234 | 133800 | 303 | 267600 | 0 | 316 | 400 | 1 | 32 | 4.561 |

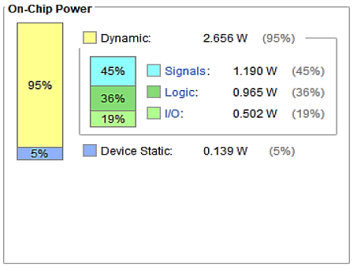
Table 4.2 summarizes the hardware resource utilization and total power consumption of the proposed perceptron architecture for different input configurations (2, 4, 6, and 8 inputs) on the target FPGA device. In terms of hardware implementation, Look-Up Tables (LUTs) and Flip-Flops (FFs) constitute the primary logic resources used to realize the combinational and sequential components of the design. LUTs implement the combinational logic required for arithmetic operations, comparisons, and weight-update equations, whereas FFs store intermediate states, weights, and bias values throughout the training process. The utilization of Input/Output (I/O) resources increases with the number of inputs because additional FPGA pins are required to accommodate the wider parallel fixed-point data interface. In contrast, the number of Global Buffers (BUFGs) remains constant, as the design relies on a single global clock network. Furthermore, DSP utilization remains zero across all configurations, indicating that all arithmetic operations are synthesized using general-purpose logic resources rather than dedicated DSP slices. Overall, LUT and FF utilization tends to increase as the number of inputs grows, reflecting the expansion of the datapath and storage requirements, although minor variations in LUT usage are observed due to synthesis optimizations. Finally, the total power consumption increases with larger input sizes, primarily as a result of higher dynamic switching activity and increased routing complexity within the FPGA fabric.



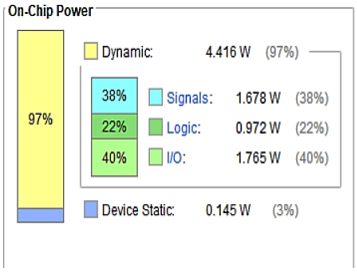

(a) Inputs = 2




(b) Inputs = 4




(c) Inputs = 6




(d) Inputs = 8

Figure 4.8 : On-Chip Power Consumption Distribution for Different Input Sizes (n = 2, 4, 6 and 8) behavioral design.

The on-chip power consumption profile of the proposed perceptron architecture is divided into **Static Power** and **Dynamic Power**. Static Power (Device Static) represents the leakage power consumed by the FPGA even when idle and remains nearly constant across all configurations, as it depends primarily on the technology node and operating conditions rather than the computational workload.

In contrast, Dynamic Power constitutes the dominant portion of the total consumption and is directly related to switching activity during active execution. It is further subdivided into three components: Signals, Logic, and I/O power.

Signals Power increases progressively with input size due to higher internal routing activity required to transfer parallel fixed-point data across the FPGA fabric. Similarly, Logic Power increases as more LUTs and registers are

activated to support arithmetic computations and weight update operations. In contrast, I/O Power remains relatively stable since external pin activity is not significantly affected by input scaling.

Overall, Dynamic Power accounts for approximately 94–95% of total consumption, while Static Power contributes only 5–6%. This distribution indicates that most of the energy is consumed during active computation rather than idle leakage, and it reflects a consistent scaling behavior of the architecture with increasing input dimensions.

### 4.5.2 Structural results and analysis

#### 4.5.2.1 Run Post-Implementation Timing Simulation

The same post-implementation timing simulation methodology described previously was applied to validate the structural model [13].

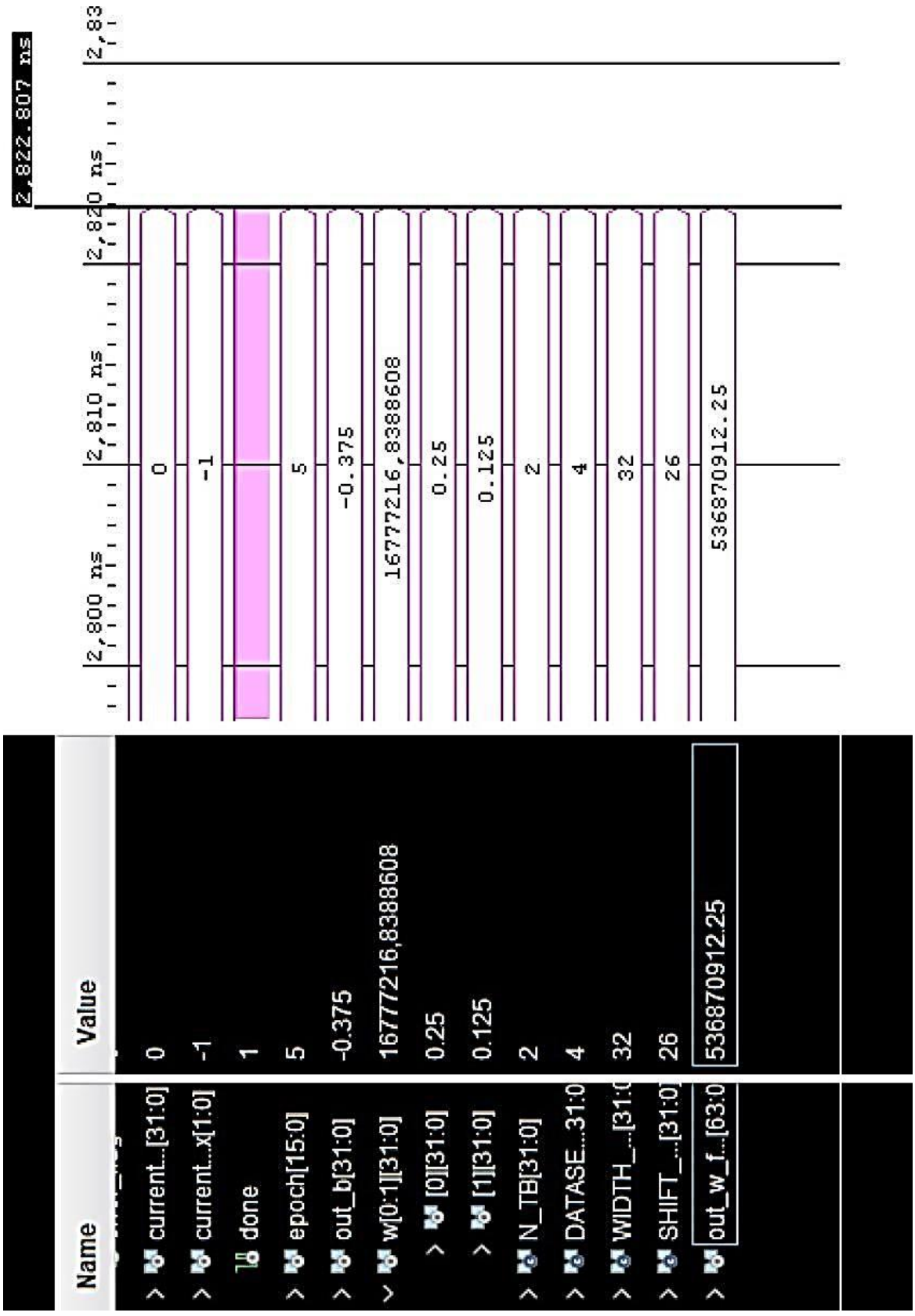



Figure 4.9: Post-implementation timing simulation results for the proposed perceptron architecture for 2 inputs.

Figure 4.10: Post-implementation timing simulation results for the proposed perceptron architecture for 4 inputs.

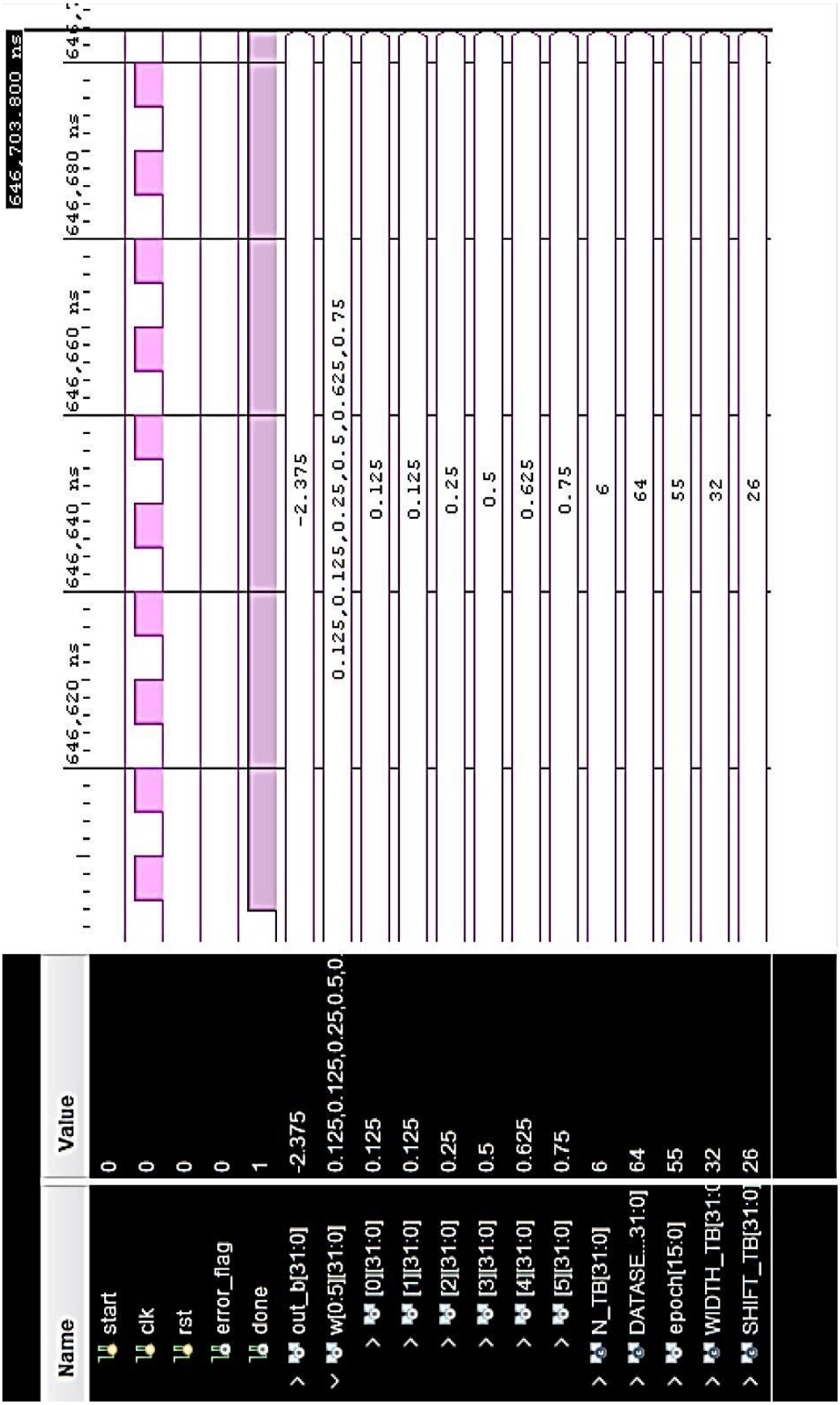



Figure 4.11: Post-implementation timing simulation results for the proposed perceptron architecture for 6 inputs.

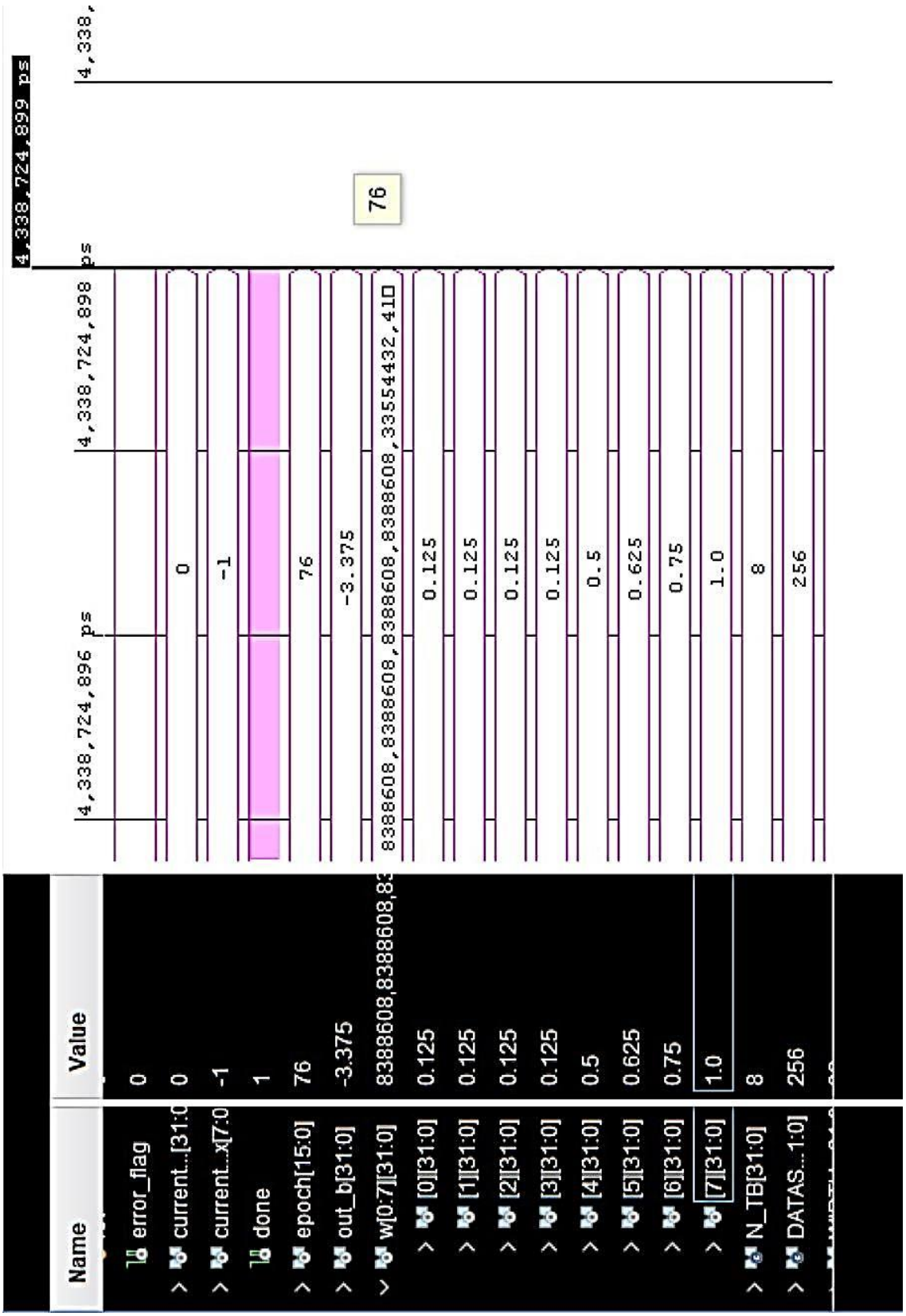



Figure 4.12: Post-implementation timing simulation results for the proposed perceptron architecture for 8 inputs.

The waveform illustrated in Figure 4.11 presents the results obtained from the post-implementation timing simulation of the proposed structural perceptron architecture following FPGA synthesis and implementation. This simulation was conducted to validate the functional correctness of the design under realistic timing constraints and hardware conditions.

The observed signals confirm the proper operation of both the training process and the control logic. Specifically, the clock signal (clk) governs the sequential execution of the system, while the reset (rst) and start signals are used to initialize and trigger the training procedure. The assertion of the done signal at the end of execution indicates the successful completion and convergence of the learning process.

The simulation results demonstrate that the perceptron required 55 training epochs to reach full convergence. Upon completion, the learned weight vector stabilized at:

**W**= [0.125, 0.125, 0.25, 0.5, 0.625, 0.75]

and the bias term converged to:

B = −2.375

Furthermore, the simulation confirms that the classification error reached zero during the final training iterations, indicating that all training samples were correctly classified. This behavior validates the correctness of both the learning rule implementation and the decision logic within the hardware design.

The total convergence time recorded during the post-implementation timing simulation was approximately 64.66 ms. These results confirm that the proposed architecture operates correctly after synthesis and implementation, and that its performance aligns with the expected behavior under realistic FPGA timing conditions.

#### 4.5.2.2 Structural Resource Utilization and Power Analysis

The hardware resource utilization and power consumption of the proposed structural perceptron architecture were evaluated for different input configurations (2, 4, and 6 inputs) on the target FPGA device.

The consolidated post-implementation results extracted from Vivado are summarized in Table 4.3.

Table 4.3: Resource Utilization and Power Consumption of the Structural Perceptron Architecture.

| **Inputs** | **LUTs** | | **Flip-flops** | | **Dsp** | **I/O** | | **BUFGs** | | **Total_ power (W)** |
|---|---|---|---|---|---|---|---|---|---|---|
| 2 | **total** | **used** | **total** | **used** | 0 | **total** | **used** | **total** | **used** | 9.905 |
| | 133800 | 75 | 267600 | 81 | | 400 | 151 | 32 | 1 | |
| 4 | 133800 | 105 | 267600 | 108 | 0 | 400 | 217 | 32 | 1 | 9.624 |
| 6 | 133800 | 141 | 267600 | 135 | 0 | 400 | 283 | 32 | 1 | 9.424 |
| 8 | 133800 | 161 | 267600 | 169 | 0 | 400 | 349 | 32 | 1 | 9.905 |

Table 4.3 summarizes the hardware resource utilization and total power consumption of the proposed structural perceptron architecture across different input dimensions.

Look-Up Tables (LUTs) and Flip-Flops (FFs) represent the primary FPGA resources utilized to implement the datapath logic, control circuitry, and storage elements [8,13]. As expected, both LUT and FF utilization increase progressively with the number of perceptron inputs due to the expansion of the datapath and the corresponding increase in storage requirements.

Input/Output (I/O) utilization also increases proportionally with the number of inputs, as additional signals are required to support wider data paths and internal routing structures. In contrast, the number of Global Clock Buffers (BUFGs) remains constant across all configurations, since the architecture operates using a single global clock network [8].

Furthermore, DSP utilization remains equal to zero for all evaluated configurations. This confirms that all arithmetic operations are implemented using general-purpose FPGA logic resources rather than dedicated DSP slices. This is primarily due to the multiplication-free perceptron architecture, where

binary inputs and shift-based parameter updates eliminate the need for hardware multipliers [8,10].

Interestingly, while hardware resource utilization increases with input size, the total power consumption shows a slight decrease. This behavior can be attributed to variations in switching activity and signal utilization efficiency within the FPGA fabric, indicating that the design maintains stable and efficient power characteristics across different configurations.

Overall, the results demonstrate that the proposed architecture exhibits scalable behavior with predictable resource growth as the input dimension increases, while maintaining efficient utilization of FPGA resources and stable power consumption. This confirms the suitability of the design for implementation on low-cost FPGA platforms with limited hardware resources.




(a) Inputs = 2

(b) Inputs = 4

(c) Inputs = 6

(d) Inputs = 8

Figure 4.13: On-Chip Power Consumption Distribution for Different Input Sizes (n = 2, 4, 6 and 8).

The power reports shown in Figure 4.7 illustrate the on-chip power consumption distribution of the proposed structural perceptron architecture for different input dimensions. The total power is divided into Device Static

Power and Dynamic Power components, while the dynamic portion is further classified into Signals, Logic, and I/O power consumption.

The obtained results indicate that Dynamic Power represents the dominant portion of the total power consumption for all evaluated configurations, accounting for approximately 98% of the overall power. In contrast, Device Static Power contributes only about 2% of the total consumption and remains nearly constant across all tested input sizes.

A detailed inspection of the reports reveals that I/O Power constitutes the largest contributor to the total dynamic power, followed by Signals Power and Logic Power. For the 2-input configuration, the power distribution consists of 8.586 W consumed by I/O resources, 0.671 W by signal routing activity, and 0.477 W by logic resources. For the 4-input configuration, the corresponding values are 7.908 W, 1.033 W, and 0.513 W, respectively. Finally, for the 6-input configuration, the measured values are 7.657 W, 1.035 W, and 0.563 W.
A noticeable trend is observed in the total power consumption, which decreases slightly as the number of inputs increases from 9.905 W for the 2-input configuration to 9.424 W for the 6-input configuration. Although this behavior may initially appear counterintuitive, it can be explained by the switching activity characteristics of the training process. For smaller input dimensions, the generated dataset contains fewer training samples, causing the finite state machine (FSM) and internal control circuitry to cycle through the training sequence more frequently within the same observation period. This results in higher switching activity and consequently higher dynamic power consumption.

#### 4.5.2.3 Scalability Limitation Due to FPGA I/O Constraints

During the scalability evaluation of the proposed Perceptron system, a clear distinction was observed between the software and hardware implementations. On the CPU platform, the developed software successfully supported input dimensions of up to 18 inputs, demonstrating the scalability of the proposed learning algorithm.

However, when transitioning to the FPGA implementation, a physical hardware limitation emerged for configurations exceeding 9 parallel inputs. Although the behavioral simulation produced functionally correct results for larger input configurations, successful hardware implementation was constrained by the available physical I/O pins on the target FPGA device. The primary bottleneck was not the availability of internal logic resources such as LUTs or Flip-Flops, but rather the limited number of external package pins required to support the parallel input interface. Consequently, the implementation stage in Vivado could not be completed successfully once the required number of I/O pins exceeded the physical capacity of the FPGA package. This observation demonstrates that the scalability limitation is imposed by the physical characteristics of the target FPGA board rather than the proposed hardware architecture or the Perceptron learning algorithm itself. Supporting larger input dimensions would therefore require alternative interface techniques, such as input multiplexing or serial communication, to reduce the number of external I/O connections [8].

### 4.5.3 Comparison Between Behavioral and Structural Implementations

A comparative analysis between the behavioral and structural implementations was conducted using a 6-input perceptron configuration in order to evaluate post-implementation hardware resource utilization and power consumption on the target FPGA device.

Table 4.4: Resource and Power Utilization Comparison Between Behavioral and Structural Implementations (6-Inputs).

| Metric | Behavioral (6Inputs) | Structural (6Inputs) | Improvement % |
|---|---|---|---|
| LUTs (Look-Up Tables) | 236 | 141 | 40.2% (Area Reduction) |
| Flip-Flops (FFs) | 263 | 135 | 48.6% (Area Reduction) |
| DSPs | 0 | 0 | 0% |

| Metric | Behavioral (6Inputs) | Structural (6Inputs) | Improvement % |
|---|---|---|---|
| Total On-Chip Power | 2.795W | 9.424 W | - |
| Logic Power (W) | 0.976 W (35%) | 0.563 W (6%) | 42.3% (Internal Efficiency) |
| Signal Power (W) | 1.302 W (46%) | 1.035 W (11%) | 20.5% (Internal Efficiency) |
| I/O Power (W) | 0.545 W (19%) | 7.657 W (83%) | - |

The results demonstrate that the structural implementation significantly optimizes resource utilization while maintaining identical functional behavior. Specifically, Look-Up Tables (LUTs) were reduced from 236 to 141, and Flip-Flops (FFs) decreased from 263 to 135. This efficiency is directly attributed to the modular architecture that explicitly decouples the Control Unit (CU) from the Datapath Unit (DU), eliminating redundant logic and enabling more efficient hardware mapping by the synthesis tool.

Regarding power characteristics, Vivado reported a higher total on-chip power consumption for the structural design (9.424 W) compared to the behavioral design (2.795 W). However, a detailed breakdown reveals that the structural architecture successfully reduced internal logic and signal power components due to minimized internal switching activity. The increase in total power was heavily dominated by the I/O power component, driven by external interface utilization and dataset transfer during timing evaluation, rather than internal architectural inefficiency.

In conclusion, the structural approach provides a more deterministic and organized architecture. By partitioning the control and datapath sections and incorporating synchronization mechanisms (e.g., the WAIT_SUM state), it achieves superior resource efficiency and enhanced timing reliability, making it highly practical for physical FPGA deployment.

### 4.5.4 Performance Evaluation and Comparative Analysis

To evaluate the computational efficiency of the proposed perceptron system, a comparative analysis was conducted between the software simulation

environment (Python) and the hardware implementation platform (FPGA). The execution time was measured across different input configurations (2, 4, and 6 inputs) under various logical operations, including AND, OR, NAND, and NOR gates. The consolidated timing results, extracted from the post-implementation timing simulations, are summarized in Table 4.5 .

Table 4.5: Execution Time and Performance Comparison between Software and Hardware Environments.

| Inputs | Environment | AND Gate (ms) | OR Gate (ms) | NAND Gate (ms) | NOR Gate (ms) | Average Time (ms) |
|---|---|---|---|---|---|---|
| **2** | Software (Python) | 7.000 | 5.000 | 15.000 | 6.000 | 8.250 |
| **2** | Hardware (Behavioral) | 0.001985 | 0.001305 | 0.001955 | 0.001275 | 0.001630 |
| **2** | Hardware (structural) | 0.002822797 | 0.001962797 | 0.002812797 | 0.001952797 | 0.002387797 |
| **4** | Software (Python) | 15.000 | 5.000 | 18.000 | 4.000 | 10.500 |
| **4** | Hardware (Behavioral) | 0.022615 | 0.008365 | 0.022565 | 0.008315 | 0.015465 |
| **4** | Hardware (structural) | 0.036514175 | 0.013044038 | 0.036504038 | 0.013834038 | 0.02497407225 |
| **6** | Software (Python) | 87.000 | 10.000 | 71.000 | 8.000 | 44.000 |
| **6** | Hardware (Behavioral) | 0.368045 | 0.053385 | 0.367975 | 0.052315 | 0.210430 |
| **6** | Hardware (structural) | 0.6467038 | 0.0926038 | 0.6466938 | 0.0925938 | 0.3696488 |
| **8** | Software (Python) | 100.931 | 15.866 | 112.587 | 23.026 | 63.1025 |
| **8** | Hardware (Behavioral) | 2.382355 | 0.309 | 2.382265 | 0.308 | 0.809 |
| **8** | Hardware (structural) | 4.338724899 | 0.507365112 | 4,338.714899 | 0.563668859 | 1,086.031164 |

The results presented in Table 4.5 show a clear increase in execution time as the number of inputs grows in both the software and hardware environments. This behavior is expected due to the increased computational complexity and larger datapath requirements associated with higher input configurations.

The FPGA implementation also exhibits increasing execution time with larger input sizes because additional routing resources, arithmetic operations, and signal transitions are required inside the hardware fabric. However, unlike the software implementation, the FPGA maintains deterministic and parallel execution behavior through dedicated hardware logic and parallel datapaths.

Additionally, AND and NAND operations generally require longer execution times compared to OR and NOR operations, mainly due to the higher number of training iterations needed for perceptron convergence.

Overall, the results demonstrate the scalability of the proposed FPGA architecture while highlighting the impact of input dimensionality on execution performance.

# CHAPTER 5
# CONCLUSION

## 5.1 Overview

This section presents the conclusion of the project. In addition, the feature work will be discussed.

## 5.2 Conclusion

This research has successfully demonstrated the design and implementation of a perceptron-based learning system on both CPU and FPGA platforms. The study focused on transforming the perceptron learning algorithm into an efficient hardware architecture using a structural design approach.

The CPU-based implementation provided a flexible environment for validating the learning algorithm, generating datasets, and verifying convergence behavior. Through iterative training, the perceptron model achieved correct classification with zero final error, confirming the correctness of the implemented learning rule.

On the hardware side, the FPGA implementation translated the algorithm into a deterministic and parallel architecture composed of a Control Unit (FSM) and a Datapath Unit. The use of fixed-point representation significantly reduced computational complexity while maintaining sufficient numerical precision. Additionally, the adoption of a shift-based learning rate (0.125) eliminated the need for hardware multipliers, resulting in efficient utilization of FPGA resources.

The post-implementation results demonstrated that the structural design achieved significant optimization in terms of resource utilization compared to the behavioral approach, with notable reductions in LUTs and Flip-Flops. Furthermore, the timing analysis confirmed that the FPGA implementation provides substantial speedup over the CPU, highlighting the advantage of hardware acceleration for machine learning tasks.

Although the total power consumption appeared higher in the structural design, detailed analysis revealed that this increase was primarily due to I/O activity rather than internal inefficiencies. In contrast, internal logic and signal power were effectively reduced, indicating an optimized computational architecture.

Overall, the proposed system validates the feasibility and efficiency of implementing perceptron-based learning on FPGA platforms. The results confirm that FPGA-based solutions offer a scalable, high-performance alternative to traditional CPU implementations, particularly for real-time and resource-constrained applications.

## 5.3 Future works

Several potential directions can be explored to further enhance the performance and efficiency of the proposed structural perceptron architecture implemented on an FPGA. In hardware-oriented designs, critical optimization objectives include minimizing area utilization and reducing power consumption.

Efficient resource utilization directly contributes to lowering implementation cost, while reducing power consumption is essential for improving thermal performance and enabling deployment in energy-constrained systems such as embedded and battery-powered applications.

In this work, a structural modeling approach was adopted to achieve better control over hardware resource allocation and to eliminate redundant logic observed in behavioral implementations. Future work may further refine this approach by exploring advanced optimization techniques such as pipelining and parallel processing to improve throughput and reduce latency. Additionally, deeper analysis of power consumption, particularly the dominant I/O power observed in the current design, can lead to more efficient interface design and data transfer mechanisms.

Another important direction is extending the current single-layer perceptron model to more complex neural architectures, such as multi-layer perceptrons (MLPs), while maintaining efficient hardware mapping. This includes

investigating scalable design methodologies that support larger input dimensions and more complex learning tasks without significantly increasing hardware cost.

Furthermore, exploring alternative numerical representations, such as optimized fixed-point formats or hybrid precision techniques, may provide a better trade-off between accuracy and hardware efficiency. Finally, integrating the proposed architecture into real-time applications and testing it on physical FPGA platforms will provide deeper insights into its practical performance, reliability, and scalability in real-world scenarios.

# CHAPTER 6
# REFERENCES


[1] 1H. K. Ali and E. M. Zeki, "Design an artificial neural network using an FPGA," *Int. J.* Comput. Sci. Netw. Secur., vol. 10, no. 8, p. 88, 2010.

[2] 2C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY, USA: Springer, 2006.

[3] A. Géron, Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2022.

[4] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[5] C. R. Harris et al., “Array programming with NumPy,” Nature, vol. 585, no. 7825, pp. 357–362, 2020.

[6] S. Haykin, Neural Networks and Learning Machines, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2009.

[7] S. Marsland, Machine Learning: An Algorithmic Perspective, 2nd ed. Boca Raton, FL, USA: CRC Press, 2015.

[8] U. Meyer-Baese and A. Meyer-Baese, Digital Signal Processing with Field Programmable Gate Arrays, 4th ed. Berlin, Germany: Springer, 2014.

[9] E. Nurvitadhi et al., "Can FPGAs beat GPUs in accelerating next-generation deep neural networks?" in Proc. 2017 ACM/SIGDA Int. Symp. Field-Programmable Gate Arrays, 2017, pp. 5–14.

[10] K. K. Parhi, VLSI Digital Signal Processing Systems: Design and Implementation. New York, NY, USA: John Wiley & Sons, 1999.

[11] S. Raschka, Python Machine Learning. Birmingham, UK: Packt Publishing, 2015.

[12] F. Rosenblatt, “The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain,” Psychological Review, vol. 65, no. 6, pp. 386–408, 1958.

[13] C. H. Roth, L. K. John, and B. K. Lee, Digital Systems Design Using Verilog, 1st ed. Boston, MA, USA: Cengage Learning, 2016.

[14] T. Zhao, "FPGA-Based Machine Learning: Platforms, Applications, Design Considerations, Challenges, and Future Directions," Highlights in Science, Engineering and Technology, vol. 62, pp. 96–101, 2023.